

Depth-Progressive Monotonic Learning without Global Backpropagation

Chenhao Ye^{1 2 3} Rongguang Ye¹ Yuchao Zhang² Ming Tang¹

Abstract

Backpropagation (BP) remains the dominant training paradigm for deep neural networks, yet its reliance on global gradient propagation fundamentally induces update locking problem, enforcing strong inter-layer dependencies in parameter updates. To address this limitation, we propose Depth-progressive Monotonic Learning (DMoL), a training scheme that assigns layer-wise local belief objectives and incrementally refines them across network depth, enabling unlocked parameter updates. As a result, DMoL supports dynamic modification of network depth during training, adapting to available compute and device resources while maintaining stable optimization. We provide theoretical guarantees that layer-wise local belief objectives improve monotonically with increasing depth and converge exponentially. Empirically, DMoL consistently matches or outperforms BP across diverse tasks, yielding a 4.3% accuracy gain on CIFAR-100, mitigating over-smoothing in deep graph neural networks (+37.5% on Cora), and reducing the final loss by over 35% in diffusion model training, highlighting its robustness and flexibility as an alternative to BP. The code is publicly available at: <https://github.com/yehAlbert/DMoL>.

1. Introduction

Backpropagation (BP) has served as the dominant training algorithm for deep neural networks for over three decades, enabling remarkable advances across vision (Simonyan & Zisserman, 2014; Dosovitskiy, 2020), language (Devlin

¹Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China ²School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, China ³School of Internet of Things, Nanjing University of Posts and Telecommunications, Nanjing, China. Correspondence to: Yuchao Zhang <yczhang@bupt.edu.cn>, Ming Tang <tangm3@sustech.edu.cn>.

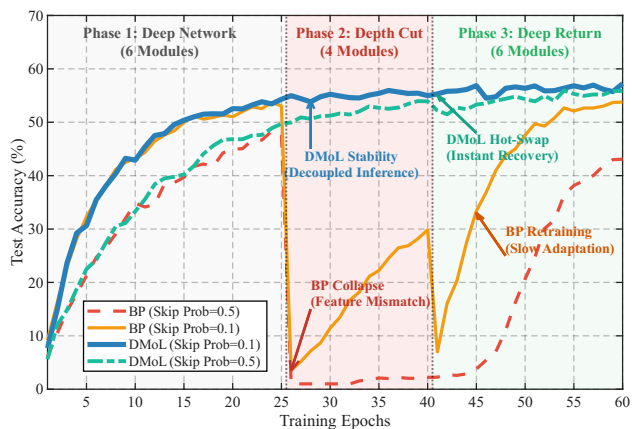


Figure 1. **Structural Fragility Test under Stochastic Skipping.** We train models with random module skipping (Phase 1) and simulate dynamic removal of intermediate modules (Phase 2), and reinstate the removed modules (Phase 3). BP suffers catastrophic collapse when depth is altered. In contrast, our proposed DMoL exhibits strong stability and instant recovery.

et al., 2019; Chowdhery et al., 2023), and generative modeling (Lipman et al., 2023). Despite its strong performance, BP faces two fundamental limitations. First, it is *biologically implausible* (Bartunov et al., 2018): each layer must have precise knowledge of the next layer’s weights to compute its backward error, which cannot be realized in natural neural systems (Ahmad et al., 2020). Second, BP enforces *update locking* (Huo et al., 2018), meaning that each layer must wait for gradients from all subsequent layers before updating its parameters. These constraints prevent existing neural networks from achieving biologically plausible local updates and efficient parallel computation, especially in large-scale systems. Most approaches—such as Hebbian learning (Journé et al., 2023) and the forward-forward algorithm (Hinton, 2022)—seek to overcome one or both of these limitations, yet they remain unable to match the performance of standard BP.

Beyond biological implausibility and update locking, we identify another fundamental limitation of BP-trained networks: *structural fragility*. As shown in Fig. 1, BP-trained networks are structurally rigid, such that removing intermediate layers during training typically results in catastrophic failure. This indicates that representations in each layer are highly entangled with other layers, preventing layers from

functioning meaningfully in isolation. Such rigidity limits its flexibility in real-world scenarios—for instance, when training large-scale models or datasets, available compute resources may vary dynamically. In these cases, the inability to adjust network depth can severely constrain efficiency and scalability. Given these limitations, a natural research question arises:

Can we design a training paradigm that overcomes the three fundamental limitations of standard BP while maintaining comparable performance?

Addressing this question is highly challenging. First, resolving update locking and biological plausibility inevitably creates discrepancies between estimated local updates and true local updates, making it difficult to update layers accurately without compromising overall performance. Second, mitigating structural fragility requires decoupling each layer’s training, so that updates do not depend on other layers, which is non-trivial to achieve while preserving accurate error propagation.

In this work, we introduce **Depth-Progressive Monotonic Learning (DMoL)**, a training framework that overcomes the three limitations by assigning each layer a local belief objective and enabling cooperative incremental refinement across network depth. Specifically, each layer maintains a probabilistic belief over the target labels and updates it using a *local cooperative objective*, which we introduce to jointly enforce alignment with task targets and consistency with the preceding layer through a combination of task and distillation losses. As a result, DMoL allows dynamic adjustment of network depth during training—layers can be added or removed as needed—enabling flexible adaptation to varying compute resources without compromising training stability. Moreover, we provide a theoretical guarantee showing that this cooperative refinement process achieves monotonic improvement with depth under mild conditions and converges exponentially.

In summary, our contributions are as follows:

- We propose Depth-Progressive Monotonic Learning (DMoL), which assigns each layer a local cooperative objective to enable fully decoupled layer-wise updates via module-restricted backpropagation, and naturally supports dynamic network depth, allowing layers to be added or removed during training to adapt to varying compute resources.
- We provide theoretical guarantees for the cooperative refinement process, proving monotonic improvement with increasing network depth and exponential convergence under mild conditions.
- Empirically, DMoL demonstrates superior performance over standard BP on multiple benchmarks,

achieving a 4.3% higher accuracy on CIFAR-100 and reducing over-smoothing by 37.5% in deep graph neural networks on Cora.

2. Related Work

We position DMoL within prior efforts to overcome the limitations of BP by organizing existing work around three recurring challenges: biological plausibility, gradient dependency, and alternative learning signals.

Biological Plausibility. A long-standing critique of BP concerns its reliance on strict weight symmetry problem (Bartunov et al., 2018), which are difficult to reconcile with biological learning mechanisms. Early biologically inspired learning rules, such as Hebbian learning (Flesch et al., 2023), emphasize purely local synaptic updates but struggle to scale to deep networks and complex tasks (Journé et al., 2023). Feedback alignment (Lillicrap et al., 2014) and direct feedback alignment (Nøkland, 2016) relax the weight symmetry constraint by using fixed random feedback matrices to propagate global error signals. However, their performance typically lags behind BP, and they are most effective in shallow networks or MLP-style architectures, making them difficult to extend to modern deep models. In contrast, our proposed DMoL confines updates to local modules, thereby eliminating the global weight symmetry constraint, and also achieves better performance to BP.

Gradient Dependency. Another major line of work seeks to alleviate the update locking problem (Huo et al., 2018) by approximating gradients rather than propagating them exactly. We discuss gradient approximation from two complementary perspectives: *global* gradient approximation and *local* gradient approximation. Global gradient approximation methods, including zero-order optimization (Ghadimi & Lan, 2013), evolutionary strategies (Qiu et al., 2025), and equilibrium propagation (Scellier & Bengio, 2017), estimate learning signals at the network level. While conceptually appealing, these approaches often incur substantial computational overhead or require iterative convergence processes that limit scalability. Local gradient approximation methods (Nøkland & Eidnes, 2019) aim to decouple layer updates by predicting gradients at intermediate layers. Decoupled neural interfaces (Jaderberg et al., 2017) train auxiliary models to estimate gradients from higher layers, addressing update locking problem. Though effective, these methods introduce additional learnable modules, increasing model size and training overhead. Other approaches aim to improve hardware utilization via pipeline parallelism, such as GPipe (Huang et al., 2019), but they remain fundamentally bound by global update locking and sequential gradients. Alternatively, Neighbor Self-Knowledge Distillation (NSKD) (Liang et al., 2024) improves representations via inter-layer distillation but still relies on full end-to-end BP, thus failing

to resolve the update locking problem.

Our proposed DMoL departs from both paradigms. Rather than approximating gradients—globally or locally—it eliminates inter-module gradient transport altogether by replacing the single global objective with a set of cooperative local objectives. This design avoids additional gradient-predicting modules and incurs minimal parameter overhead.

Alternative Learning Signals. A broad class of methods alleviates the limitations of BP by reformulating the underlying optimization objective (Sun et al., 2025; Chen et al., 2025; Aghagolzadeh & Ezoji, 2025; Ren & Li, 2024). For example, the forward-forward algorithm (Hinton, 2022) introduces a local goodness objective based on separate forward passes for positive and negative data, while equilibrium propagation (Scellier & Bengio, 2017) derives learning signals from energy-based dynamics converging to fixed points. Other methods, such as PETRA (Rivaud et al., 2025) and NoProp (Li et al., 2025), further depart from standard BP by redesigning training objectives to improve parallelism or eliminate gradient propagation altogether. Difference Target Propagation (DTP) (Lee et al., 2015) and Predictive Coding (PC) (Whittington & Bogacz, 2017) seek to replace BP by relying on inverse mappings or iterative forward-backward interactions, which still induce structural dependencies and update locking. Associated Learning (Wu et al., 2021) proposes an alternative but requires heavy feature-space matching with auxiliary autoencoders. Recently, Stochastic Layer-wise Learning (SLL) (Yin & Corradi, 2025) adopts a probabilistic perspective similar to DMoL; however, SLL relies on task-agnostic random projections and assumes a fixed-depth execution, lacking DMoL’s task-adaptive semantic alignment and structural robustness under dynamic depth. A complementary line of work augments the optimization objective by using layer-wise supervision. Knowledge distillation (Hinton et al., 2015) leverages intermediate representations of a teacher to guide a student, and has been extended to layer-wise self-distillation schemes (Furlanello et al., 2018; Gurioli et al., 2025; Xu et al., 2025; Liu et al., 2024). Unlike prior methods, our proposed DMoL allows layers to be added or removed during training in response to available compute, while ensuring that the cooperative objective exhibits monotonic improvement as network depth increases.

3. Depth-Progressive Monotonic Learning

In this section, we introduce the Depth-Progressive Monotonic Learning (DMoL) framework. We first define a layer-wise local belief formulation that reinterprets the outputs of intermediate layers as standalone predictive signals. We then describe a depth-wise cooperative refinement process, through which deeper layers progressively refine representations while ensuring monotonic performance improvement

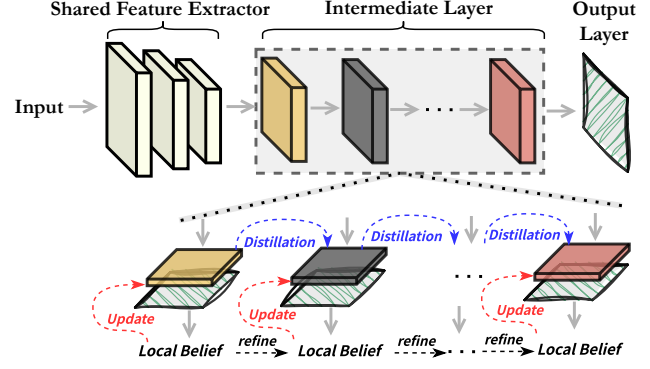


Figure 2. **The proposed DMoL framework.** Each intermediate layer maintains a local belief under the task supervision and incrementally refines the belief from the preceding layer via knowledge distillation.

with increasing depth. Finally, we describe an asynchronous-ready training algorithm that eliminates backward locking.

3.1. Local Belief Representation

At the core of DMoL is an explicit belief representation maintained at each depth of the network. We consider a deep model composed of a shared feature extractor $\mathcal{F}(\cdot; \theta_{\mathcal{F}})$ and a sequence of L intermediate layers $\{f_1, \dots, f_L\}$, as shown in Fig. 2.

To facilitate depth-wise prediction, DMoL enforces explicit label-space alignment at every layer. Specifically, layer i parameterized with f_i is augmented with a lightweight projection head¹ that maps its representation directly to a categorical distribution over $\mathcal{Y} = \{1, \dots, m\}$. This distribution represents the module’s belief regarding the target class.

For an input $\mathbf{x}$, the belief refinement process begins with an uninformative initial belief $\mathbf{p}_0$, set as a uniform distribution over $\mathcal{Y}$. The shared feature extractor first computes global features $\mathbf{Z} = \mathcal{F}(\mathbf{x}; \theta_{\mathcal{F}})$, which are provided to all subsequent modules. Each module f_i then incrementally updates the incoming belief $\mathbf{p}_{i-1}$ using both the shared features $\mathbf{Z}$ and its own parameters θ_i :

$$\mathbf{p}_i = f_i(\mathbf{p}_{i-1}, \mathbf{Z}; \theta_i), \quad i = 1, \dots, L. \quad (1)$$

This defines a belief-centric interface between layers: instead of propagating raw feature maps, each layer refines a probabilistic belief about the target, gradually improving predictions as depth increases. The final output of the network is given directly by the terminal belief $\mathbf{p}_L$.

¹In our implementation, the projection head is a single linear layer. Since the number of classes is typically much smaller than the feature dimensionality, the number of parameters in the projection layer is small.

3.2. Cooperative Monotonic Learning

To enable fully decoupled training, we design a *local cooperative objective* $\mathcal{L}_i$ for each module f_i , with two complementary goals: (i) ensure that the module’s belief $\mathbf{p}_i$ accurately predicts the ground-truth label y , and (ii) encourage incremental refinement of the incoming belief $\mathbf{p}_{i-1}$ from the previous layer.

Formally, we define the loss function for the parameters θ_i as a convex combination of two Kullback-Leibler (KL) divergences:

$$\mathcal{L}_i(\theta_i) = \underbrace{\alpha D_{\text{KL}}(\mathbf{p}_i \| \mathbf{p}_y)}_{\text{task-alignment}} + \underbrace{(1 - \alpha) D_{\text{KL}}(\mathbf{p}_i \| \text{sg}(\mathbf{p}_{i-1}))}_{\text{consistency}}, \quad (2)$$

where $\mathbf{p}_y$ is the smoothed target distribution (see Appendix C.1), $\alpha \in (0, 1)$ is the balancing coefficient, and $\text{sg}(\cdot)$ denotes the stop-gradient operator.

A critical design choice in Eq. (2) is the direction of the divergence. Unlike standard cross-entropy which minimizes $D_{\text{KL}}(\mathbf{p}_y \| \mathbf{p}_i)$ and effectively seeks an *arithmetic* mixture of distributions, we minimize $D_{\text{KL}}(\mathbf{p}_i \| \cdot)$. As derived in Section 4, placing the prediction $\mathbf{p}_i$ as the first argument mathematically forces the optimal solution to be a geometric interpolation between the prior $\mathbf{p}_{i-1}$ and the target $\mathbf{p}_y$. This geometric property is essential: it corresponds to performing additive updates in the logit space (similar to boosting or residual connections), thereby enabling the model to theoretically achieve exponential error decay across layers.

The first term (*task-alignment*) thus injects new supervisory information into the logits, while the second term (*consistency*) acts as a structural anchor, ensuring the refinement builds strictly upon the predecessor’s belief.

From an optimization perspective, our proposed local cooperative objective provides each layer with an independent optimization signal. Instead of global BP, DMoL utilizes module-restricted backpropagation, breaking inter-layer backward dependencies while confining the chain rule to isolated submodules. This design simultaneously addresses the global weight-transpose issue (i.e., biological implausibility) and mitigates update locking. Moreover, the cooperative interaction between the task-alignment and consistency terms ensures that the network remains stable even when some layers are removed during training, effectively alleviating structural fragility.

3.3. Decoupled Optimization Procedure

The decoupled design of our local cooperative objective leads to an optimization procedure that is explicitly separated into two phases within training process. The training procedure removes inter-layer backward dependencies and enables parallel parameter updates, as summarized in Algo-

Algorithm 1 DMoL Training Procedure

```

1: Input: Minibatch  $B = \{(\mathbf{x}_k, y_k)\}$ , parameters  $\theta_{\mathcal{F}}$ ,
    $\theta_{\text{modules}} = \{\theta_i\}_{i=1}^L$ , and consistency weight  $\alpha$ .
2: // Phase 1: Generate Teacher Beliefs
3:  $\mathbf{Z}_{\text{det}} \leftarrow \mathcal{F}(B_{\mathbf{x}}; \theta_{\mathcal{F}})$  with gradients disabled;
4:  $\mathcal{P}_{\text{teachers}} \leftarrow [\text{Uniform}(\mathcal{Y})]$ ;
5: for  $i = 1$  to  $L - 1$  do
6:    $\mathbf{p}_i \leftarrow f_i(\mathcal{P}_{\text{teachers}}[i - 1], \mathbf{Z}_{\text{det}}; \theta_i)$ ;
7:   Append  $\mathbf{p}_i$  to  $\mathcal{P}_{\text{teachers}}$ ;
8: end for
9: // Phase 2: Parallel Local Updates
10:  $\mathbf{g}_{\mathcal{F}} \leftarrow 0$ ;
11:  $\mathbf{Z} \leftarrow \mathcal{F}(B_{\mathbf{x}}; \theta_{\mathcal{F}})$  with gradients enabled;
12: for  $i = 1$  to  $L$  in parallel do
13:   Retrieve teacher  $\mathbf{p}_{i-1}$  from  $\mathcal{P}_{\text{teachers}}$ ;
14:    $\mathbf{p}_i \leftarrow f_i(\mathbf{p}_{i-1}, \mathbf{Z}; \theta_i)$ ;
15:   Compute  $\mathcal{L}_i$  via Eq. (2);
16:   Compute gradients:  $\mathbf{g}_i \leftarrow \nabla_{\theta_i} \mathcal{L}_i$  and  $\mathbf{g}_{\mathcal{F},i} \leftarrow \nabla_{\theta_{\mathcal{F}}} \mathcal{L}_i$ ;
17:   Atomic:  $\mathbf{g}_{\mathcal{F}} \leftarrow \mathbf{g}_{\mathcal{F}} + \mathbf{g}_{\mathcal{F},i}$ ;
18: end for
19: Update  $\theta_{\mathcal{F}}$  using  $\mathbf{g}_{\mathcal{F}}$  and all  $\{\theta_i\}$  using  $\{\mathbf{g}_i\}$ ;
    
```

rithm 1.

Phase 1: Teacher Belief Generation. As shown in Algorithm 1 (lines 3–8), the first phase generates the teacher beliefs required for local optimization. Specifically, we perform a forward pass through the network, sequentially computing and caching the intermediate beliefs $\mathbf{p}_0, \dots, \mathbf{p}_{L-1}$. Each $\mathbf{p}_{i-1}$ serves as the teacher signal for the i -th layer in the subsequent update phase.

Importantly, this phase does not construct a computation graph and therefore incurs minimal memory overhead. In distributed or pipelined systems, teacher generation can be executed asynchronously or overlapped with other computations, making it lightweight in practice.

Phase 2: Parallel Local Updates. Once the teacher beliefs are fixed, parameter updates for all layers become fully decoupled. As illustrated in Algorithm 1 (lines 10–18), we first recompute the shared representation $\mathbf{Z} = \mathcal{F}(\mathbf{x}; \theta_{\mathcal{F}})$. Each layer then independently evaluates its local objective $\mathcal{L}_i$ using the cached teacher belief $\mathbf{p}_{i-1}$ and the current features $\mathbf{Z}$. Since the teacher signals act as fixed regularization targets in the local objectives, gradients are confined to each module, i.e., a single intermediate layer together with its projection head. Consequently, the gradients $\nabla_{\mathcal{L}_1}, \dots, \nabla_{\mathcal{L}_L}$ can be computed in parallel, completely eliminating update locking. Finally, gradients with respect to the shared extractor $\theta_{\mathcal{F}}$ are accumulated across modules and applied in a single synchronized update step (line 19).

4. Theoretical Analysis

In this section, we analyze the convergence behavior of DMoL. To facilitate the analysis, we abstract the optimization from the parameter space θ_i to the function space of probability distributions. We define the local objective functional $\mathcal{L}_i(\mathbf{p})$ for a candidate belief $\mathbf{p}$, given a target $\mathbf{p}_y$ and a fixed prior $\mathbf{p}_{i-1}$, as:

$$\mathcal{L}_i(\mathbf{p}) \triangleq \alpha D_{\text{KL}}(\mathbf{p} \parallel \mathbf{p}_y) + (1 - \alpha) D_{\text{KL}}(\mathbf{p} \parallel \mathbf{p}_{i-1}). \quad (3)$$

The training goal for module f_i described in Eq. (2) is equivalent to finding parameters θ_i such that the output $\mathbf{p}_i$ minimizes this functional.

4.1. Convergence and Stability

We first characterize the behavior of the cooperative refinement process in two scenarios: an idealized setting where modules perfectly minimize their objectives, and a practical setting where modules only achieve bounded descent.

Scenario 1: Exact Minimization Dynamics. This scenario abstracts away optimization imperfections to reveal the fundamental convergence rate of the belief refinement cascade.

Assumption 4.1 (Optimal Local Updates). Each module f_i is sufficiently expressive and optimized such that its output $\mathbf{p}_i$ is the unique minimizer of $\mathcal{L}_i(\mathbf{p}_i)$. We denote this optimality as $\mathbf{p}_i = \arg \min_{\mathbf{p}} \mathcal{L}_i(\mathbf{p})$, where $\arg \min_{\mathbf{p}}$ implies pointwise equality of the probability mass functions.

Under Assumption 4.1, the sequence of beliefs collapses to a geometric interpolation between the initial uniform prior and the target.

Proposition 4.2 (Geometric Error Decay). *Given an initial uniform belief $\mathbf{p}_0$ and Assumption 4.1, the belief $\mathbf{p}_i$ at depth i satisfies:*

$$\mathbf{p}_i(k) \propto \mathbf{p}_0(k)^{(1-\alpha)^i} \mathbf{p}_y(k)^{1-(1-\alpha)^i}, \quad \forall k \in \mathcal{Y}. \quad (4)$$

The belief $\mathbf{p}_i$ converges pointwise to the target $\mathbf{p}_y$ as $i \rightarrow \infty$. The convergence rate is governed geometrically by the factor $(1 - \alpha)$ (Proof in Appendix A.1).

Proposition 4.2 establishes that even conservative local updates (governed by α) compound exponentially to concentrate probability mass on the correct label. The factor $(1 - \alpha)$ explicitly dictates the speed of this information refinement.

Scenario 2: Relaxed Descent Dynamics. In practice, perfect minimization is unattainable due to finite data and stochastic optimization. We relax Assumption 4.1 to a weaker condition: modules effectively reduce their local loss.

Proposition 4.3 (Global Performance Bound). *Given the local descent condition $\mathcal{L}_i(\mathbf{p}_i) \leq \mathcal{L}_i(\mathbf{p}_{i-1})$, the KL divergence from the final belief $\mathbf{p}_L$ to the target is bounded by the initial divergence minus the cumulative consistency gains:*

$$D_{\text{KL}}(\mathbf{p}_L \parallel \mathbf{p}_y) \leq D_{\text{KL}}(\mathbf{p}_0 \parallel \mathbf{p}_y) - \frac{1 - \alpha}{\alpha} \sum_{i=1}^L D_{\text{KL}}(\mathbf{p}_i \parallel \mathbf{p}_{i-1}). \quad (5)$$

(Proof in Appendix A.2).

This result guarantees that local improvements translate directly to global non-degradation. Furthermore, for the global error to remain bounded as L increases, the sum of divergences $\sum_{i=1}^{\infty} D_{\text{KL}}(\mathbf{p}_i \parallel \mathbf{p}_{i-1})$ must converge, implying that $D_{\text{KL}}(\mathbf{p}_i \parallel \mathbf{p}_{i-1}) \rightarrow 0$. This indicates that the belief sequence becomes asymptotically stationary, ensuring a stable flow of information even in very deep networks.

4.2. Parallelism and Computational Efficiency

A fundamental advantage of DMoL is the removal of update locking, which enables the parallelization of the computationally expensive backward passes. We analyze the theoretical implications of this architectural shift on a system with P parallel devices ($P \geq L$). Let $C_f^{(i)}$ and $C_b^{(i)}$ denote the forward and backward computation costs for module i , and A_i its activation memory cost.

Training Latency: In standard BP, the sequential dependency forces the total time to be the sum of all operations: $T_{\text{BP}} = \sum_{i=1}^L (C_f^{(i)} + C_b^{(i)})$. In DMoL, while the teacher generation phase is sequential, the gradient updates run in parallel. The time complexity becomes $T_{\text{DMoL}} \approx \sum_{i=1}^L C_f^{(i)} + \max_i C_b^{(i)}$. By parallelizing the backward pass, DMoL shifts the complexity from scaling with the *sum* of backward costs to the *maximum* single-module cost.

Peak Device Memory: BP requires storing activations for the entire network path to compute gradients, leading to a peak memory usage of $M_{\text{BP}} \approx \sum_{i=1}^L A_i$. In contrast, DMoL decouples the computation graph; each device i only requires activations for its specific module f_i to compute $\nabla_{\theta_i} \mathcal{L}_i$. Consequently, the per-device peak memory is reduced to $M_{\text{DMoL}} \approx \max_i A_i$.

This analysis confirms that DMoL fundamentally transforms the scaling laws of training deep networks. The decoupling of modules converts network depth from a latency bottleneck into a parallelizable workload.

5. Experiments

In this section, we first detail the experimental setup, including datasets, baselines, and implementation. We then evaluate DMoL across vision, graph, and generative tasks,

and further discuss its structural stability, parallel efficiency, depth scalability, and performance across different network architectures. Appendix D provides additional analyses, including the effect of the balancing coefficient α , feature space visualizations under DMoL, and comparative performance under noisy scenarios.

5.1. Experimental Settings

Baselines: We categorize baselines into three groups: (I) *Global Error Propagation*, represented by standard BP; (II) *Gradient Approximation*, including Feedback Alignment (FA) (Nøkland, 2016) and Direct Feedback Alignment (DFA) (Nøkland, 2016); and (III) *Local Supervision*, covering Forward-Forward (FF) (Hinton, 2022), NoProp (Li et al., 2025), Decoupled Greedy Learning (DGL) (Belilovsky et al., 2020), and HSIC Bottleneck learning (Ma et al., 2019).

Datasets. For vision tasks, we evaluate on CIFAR-10, CIFAR-100 (Krizhevsky, 2009), Tiny-ImageNet, and ImageNet (Deng et al., 2009) datasets. For graph tasks, we use standard node classification benchmarks: Cora, PubMed (Yang et al., 2016), and Actor (Pei et al., 2020). Deep Graph Convolutional Networks (GCNs) are employed to assess robustness against the over-smoothing phenomenon. For generative tasks, we focus on label-to-image generation on CIFAR-10, where each layer of the Diffusion Transformer (DiT) is supervised to predict the added noise. Detailed derivations are provided in Appendix C.2.2.

Implementation Details. To ensure a fair comparison, all methods employ identical data augmentation pipelines and backbone architectures (see Appendix C.1). Convolutional models are trained with SGD using Nesterov momentum (0.9). For ImageNet, we employ a Vision Transformer (ViT-Tiny) architecture trained using the AdamW optimizer. All models utilize a cosine annealing learning rate schedule. For alternative paradigms (e.g., FF, DGL), we adopt the hyperparameter configurations recommended in their original implementations. All reported results are averaged over three random seeds. Detailed architecture specifications and hyperparameter grids are provided in Appendix C.1.2.

5.2. Evaluation on Vision Tasks

From Table 1, three key observations emerge: **(I) Competitive performance with BP.** DMoL matches BP on CIFAR-10 (84.42% vs 84.32%) and ImageNet (63.74% vs 63.68%), indicating that decoupled local learning does not compromise accuracy on easy tasks or when scaling to large datasets. Notably, on medium-scale datasets such as CIFAR-100 and Tiny-ImageNet, DMoL surpasses BP by 4.31% and 6.86%, respectively. This improvement, particularly on medium-scale datasets, suggests that DMoL’s decoupled objectives alleviate optimization challenges that commonly

hinder standard BP in deep networks. **(II) Stability compared to gradient approximation methods.** Methods relying on approximate gradients, such as FA and DFA, exhibit rapid performance degradation as task difficulty increases. In contrast, DMoL maintains stable performance across datasets, highlighting the robustness of its local learning signals even under challenging tasks. **(III) Superior local supervision performance.** Compared with other local supervision methods, DMoL achieves the best accuracy in three out of four datasets. Fig. 3 further illustrates that DMoL converges faster than alternative local supervision methods, highlighting both its efficiency and stability. Moreover, its design allows layers to be flexibly added or removed depending on computational resources, offering practical adaptability.

5.3. Evaluation on Graph and Generative Tasks

Graph Learning. Deep GNNs often suffer from the *over-smoothing* problem, where node features converge to indistinguishable values and gradients vanish as depth increases. To evaluate DMoL in this context, we trained an 8-layer GCN on standard citation datasets. As shown in Table 2, standard BP fails catastrophically on the deep architecture (e.g., 43.86% on Cora), barely exceeding random performance. In contrast, DMoL achieves 81.40%, effectively recovering the performance of shallow state-of-the-art models. By enforcing local supervision at each layer, DMoL mitigates feature collapse and maintains informative node representations, demonstrating its ability to stabilize training in deep GNNs where standard BP struggles.

Fig. 4 visualizes the evolution of node feature manifolds in an 8-layer GCN. In BP (top), manifolds collapse into highly entangled ribbons, reflecting severe over-smoothing. In contrast, DMoL (bottom) exhibits progressive disentanglement, forming compact and separable clusters. This illustrates that DMoL leverages network depth to refine representations, turning the collapse observed in standard BP into a structured, informative feature hierarchy.

Generative Learning. We interpret the denoising process in diffusion models as an iterative refinement of the predicted noise ϵ . To evaluate DMoL in a label-to-image generation setting, we applied it to a diffusion Transformer (DiT) on CIFAR-100 and Tiny-ImageNet, where each layer receives a local supervision signal to predict the added noise. Fig. 5 reports the training Mean Squared Error (MSE). DMoL converges to a lower loss (i.e., 0.034) compared to standard BP (i.e., 0.060). This improvement reflects that the layer-wise objectives provide more direct supervision to each layer, facilitating better convergence result.

Table 1. **Classification Accuracy on Vision Benchmarks.** Comparison of DMoL against global and local training methods. The best performance among local/alternative methods is **bolded** and the second best is underlined.

Method	CIFAR-10	CIFAR-100	Tiny-ImageNet	ImageNet
<i>Global Error Propagation</i>				
Backpropagation	84.32 $\pm$ 0.45	51.81 $\pm$ 0.52	34.51 $\pm$ 0.61	<u>63.68 $\pm$ 0.21</u>
<i>Gradient Approximation (Global-Guided)</i>				
Feedback Alignment	77.03 $\pm$ 0.68	18.32 $\pm$ 0.75	6.39 $\pm$ 0.42	1.32 $\pm$ 0.28
Direct Feedback Alignment	73.53 $\pm$ 0.32	36.78 $\pm$ 0.28	26.58 $\pm$ 0.92	15.48 $\pm$ 0.72
<i>Local Supervision (Gradient-Free / Local Gradient)</i>				
NoProp	84.18 $\pm$ 0.33	35.69 $\pm$ 0.59	2.80 $\pm$ 0.25	0.27 $\pm$ 0.14
Forward-Forward	34.10 $\pm$ 0.81	9.13 $\pm$ 0.48	2.13 $\pm$ 0.19	0.80 $\pm$ 0.17
Decoupled Greedy Learning	84.24 $\pm$ 0.14	54.31 $\pm$ 0.37	<u>38.13 $\pm$ 0.47</u>	36.30 $\pm$ 0.27
HSIC Bottleneck	85.21 $\pm$ 0.29	<u>55.18 $\pm$ 0.41</u>	35.61 $\pm$ 0.38	33.62 $\pm$ 0.35
DMoL (Ours)	<u>84.42 $\pm$ 0.35</u>	56.12 $\pm$ 0.38	41.37 $\pm$ 0.45	63.74 $\pm$ 0.26

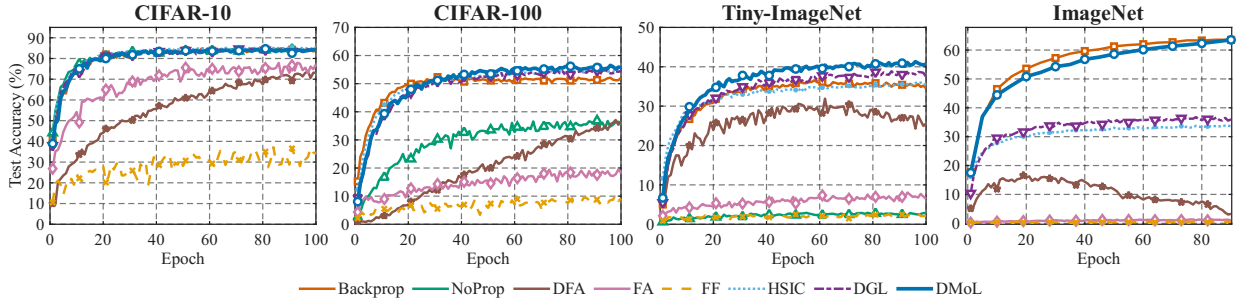


Figure 3. **Convergence Results.** Test accuracy curves on CIFAR-10, CIFAR-100, Tiny-ImageNet, and ImageNet.

Table 2. **8-Layer GCN Performance.** Test Accuracy (%) reported as Mean $\pm$ Std. Δ indicates improvement over BP.

Dataset	Backpropagation	DMoL (Ours)	Δ
Cora	43.86 $\pm$ 16.5	81.40 $\pm$ 0.70	$\uparrow$ 37.54
PubMed	66.22 $\pm$ 5.14	77.38 $\pm$ 0.62	$\uparrow$ 11.16
Actor	25.46 $\pm$ 1.31	27.78 $\pm$ 0.64	$\uparrow$ 2.32

5.4. Discussion

Structural Stability Analysis. To evaluate structural stability under dynamic compute availability, we design two training scenarios. Scenario 1: One-third of intermediate modules are removed at epoch 25 and restored at epoch 40 (dynamic depth reconfiguration). Scenario 2: The same dynamic depth reconfiguration combined with stochastic update skipping, where parameter updates are independently disabled per module. As shown in Fig. 6, DMoL exhibits stable training behavior under both scenarios. In particular, across these two scenarios, BP experiences catastrophic performance degradation under network structure changes, while DMoL maintains stable convergence and quickly re-

covers after module restoration. This suggests that the local objective in DMoL significantly improves robustness to dynamic depth variation and structural perturbations.

Pipeline Parallelism Analysis. To simulate dynamic changes in computational resources, we create a distributed training setup in which each module runs on a separate thread or worker. Fig. 7a compares the normalized training throughput on CIFAR-100. BP’s throughput (Red) saturates quickly because Layer i must wait for the gradient from layer $i + 1$. In contrast, DMoL (Blue) scales linearly with the number of parallel workers. This confirms that DMoL effectively utilizes parallel resources by decoupling the backward pass.

Monotonic Performance Improvement with Depth. Fig. 7b reports CIFAR-100 accuracy as the number of modules increases from 4 to 20. Standard BP shows a clear decline in performance with increasing depth, whereas DMoL improves monotonically, consistently benefiting from additional layers. This behavior indicates that BP suffers from vanishing gradients in deep non-residual networks, while DMoL’s decoupled objectives maintain stable optimization and effectively leverage added depth. The results highlight

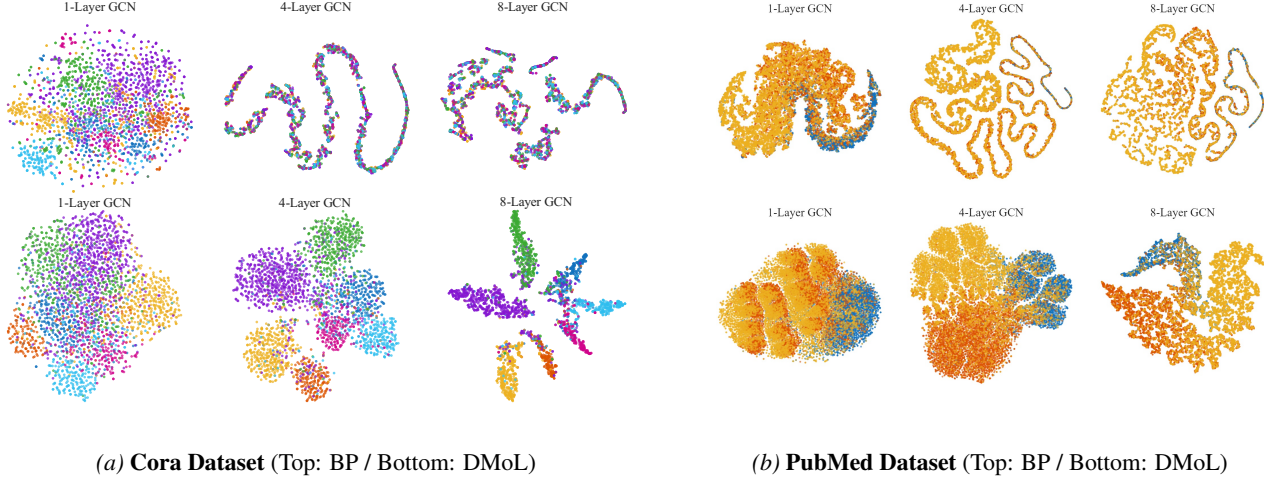


Figure 4. **Manifold Evolution via t-SNE.** **Top Row (BP):** Standard BP leads to representation collapse (over-smoothing). **Bottom Row (DMoL):** DMoL enforces progressive refinement, resulting in compact class clusters. DMoL effectively disentangles the graph structure across network depth.

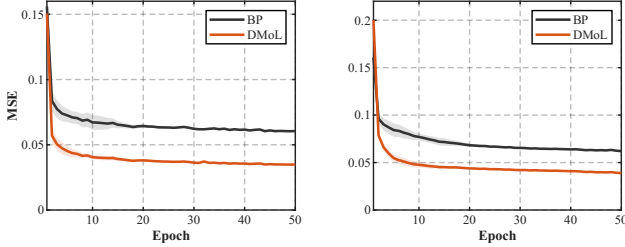


Figure 5. **DiT Training Loss.** MSE of label-to-image prediction on CIFAR-100 (left) and Tiny-ImageNet (right).

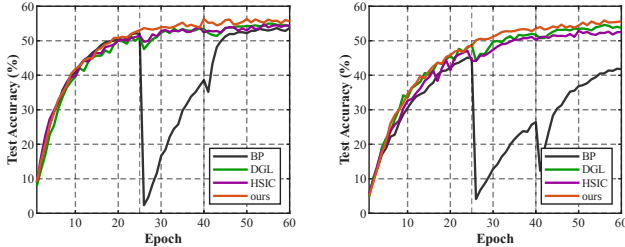
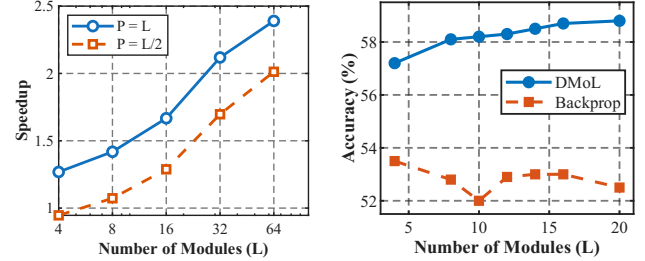


Figure 6. **Structural Fragility Test.** **Right:** Dynamic depth re-configuration, where one-third of intermediate modules are removed at epoch 25 and restored at epoch 40. **Left:** Dynamic depth re-configuration with stochastic update skipping, where parameter updates are independently disabled per module with probability 0.5 while forward computation remains unchanged.

DMoL’s robustness and stability under deeper architectures.

Architectural Adaptability. Table 3 reports DMoL’s performance across several standard backbones on CIFAR-10 and CIFAR-100. On architectures naturally suited for local module objectives, such as SimpleCNN and VGG, DMoL matches or slightly exceeds BP, demonstrating effective adaptation without degradation. These results highlight DMoL’s ability to achieve BP-competitive performance



(a) Throughput speedup over BP versus the number of parallel workers P . (b) Accuracy across network depths (L).

Figure 7. Evaluation of DMoL on CIFAR-100.

across heterogeneous network architectures without relying on global BP. For deeper architectures like ResNet-18, DMoL’s performance depends on architectural modifications: the standard ResNet-18 exhibits some mismatch with layer-wise objectives, leading to a drop compared to BP, whereas the ResNet-18* variant, adapted for DMoL, restores and even slightly surpasses BP accuracy. Notably, this flexibility extends across architectures, enabling depth modification under dynamic computational budgets while maintaining BP-competitive performance.

6. Conclusion and Future Directions

Conclusion. We proposed DMoL, a training framework that decouples deep network optimization by assigning each module a local objective, removing the sequential dependency of BP and enabling parallel updates. Theoretical analysis shows that layer-wise objectives guarantee monotonic improvement across depth and converge exponentially. Experiments demonstrate that DMoL stabilizes training in

Table 3. Full results for Architectural Generality. Accuracy (%) of DMoL vs. BP on standard backbones, reported as mean $\pm$ std.

Dataset	Backbone	BP	DMoL (Ours)
CIFAR-10	SimpleCNN	84.31 $\pm$ 0.90	84.37 $\pm$ 0.26
	VGG-11	88.21 $\pm$ 0.15	88.26 $\pm$ 0.18
	ResNet-18	95.68 $\pm$ 0.22	93.32 $\pm$ 0.17
	ResNet-18*	95.68 $\pm$ 0.22	95.87 $\pm$ 0.14
CIFAR-100	SimpleCNN	52.00 $\pm$ 0.65	54.05 $\pm$ 0.11
	VGG-11	61.01 $\pm$ 0.24	62.33 $\pm$ 0.31
	ResNet-18	78.31 $\pm$ 0.43	73.02 $\pm$ 0.38
	ResNet-18*	78.31 $\pm$ 0.43	79.16 $\pm$ 0.35

* ResNet-18* denotes ResNet-DMoL-RA (see Appendix E.2).

deep non-residual networks and generalizes beyond vision tasks, effectively mitigating over-smoothing in graphs and improving label-to-image generation. These results establish DMoL as a scalable, stable, and versatile alternative to standard BP.

Future Directions. While DMoL demonstrates strong performance across diverse tasks and architectures, its principles could be extended to even larger and more complex networks. Future work may extend DMoL framework to a broader range of tasks beyond vision, graph, and generative modeling. Another promising direction is to further decouple DMoL’s asynchronous updates into layer-wise independent forward computation, enabling parallel and efficient forward execution across layers in neural networks.

Acknowledgements

This work was supported in part by the National Natural Science Foundation of China under Grant 62202214 and Guangdong Basic and Applied Basic Research Foundation under Grant 2023A1515012819.

Impact Statement

This work introduces DMoL, a method for decoupled optimization in deep neural networks, with potential implications for improving the scalability, efficiency, and stability of large-scale models. While primarily a technical contribution, its applications span vision, graph, and generative modeling, and could facilitate more resource-efficient training in real-world machine learning systems. The approach does not introduce direct societal risks, but by enabling faster and more robust model training.

References

Aghagolzadeh, H. and Ezoji, M. Contrastive forward-forward: A training algorithm of vision transformer, 2025. URL <https://arxiv.org/abs/2502.00571>.

Ahmad, N., van Gerven, M. A., and Ambrogioni, L. GAIT-prop: A biologically plausible learning rule derived from backpropagation of error. In *Proceedings of Advances in Neural Information Processing Systems*, volume 33, pp. 10913–10923, 2020.

Bartunov, S., Santoro, A., Richards, B., Marris, L., Hinton, G. E., and Lillicrap, T. Assessing the scalability of biologically-motivated deep learning algorithms and architectures. In *Proceedings of Advances in Neural Information Processing Systems*, volume 31, 2018.

Belilovsky, E., Eickenberg, M., and Oyallon, E. Decoupled greedy learning of cnns, 2020. URL <https://arxiv.org/abs/1901.08164>.

Chen, X., Liu, D., Laydevant, J., and Grollier, J. Self-contrastive forward-forward algorithm. *Nature Communications*, 16(1):5978, 2025. doi: 10.1038/s41467-025-61037-0. URL <https://doi.org/10.1038/s41467-025-61037-0>.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of Association for Computational Linguistics: Human Language Technologies*, pp. 4171–4186, 2019.

Dosovitskiy, A. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

Flesch, T., Nagy, D. G., Saxe, A., and Summerfield, C. Modelling continual learning in humans with hebbian context gating and exponentially decaying task signals. *PLoS Computational Biology*, 19(1):e1010808, 2023.

Furlanello, T., Lipton, Z. C., Tschannen, M., Itti, L., and Anandkumar, A. Born again neural networks, 2018. URL <https://arxiv.org/abs/1805.04770>.

Ghadimi, S. and Lan, G. Stochastic first-and zeroth-order methods for nonconvex stochastic programming. *SIAM journal on optimization*, 23(4):2341–2368, 2013.

- Gurioli, A., Pennino, F., Monteiro, J., and Gabbriellini, M. ONE MODEL TO TRAIN THEM ALL: HIERARCHICAL SELF-DISTILLATION FOR ENHANCED EARLY LAYER EMBEDDINGS. In *ICLR 2025 Third Workshop on Deep Learning for Code*, 2025. URL <https://openreview.net/forum?id=FBYtU6Mcnl>.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition, 2015. URL <https://arxiv.org/abs/1512.03385>.
- Hinton, G. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2(3):5, 2022.
- Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M., Lee, H., Ngiam, J., Le, Q. V., Wu, Y., et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- Huo, Z., Gu, B., Huang, H., et al. Decoupled parallel back-propagation with convergence guarantee. In *Proceedings of International Conference on Machine Learning*, pp. 2098–2106. PMLR, 2018.
- Jaderberg, M., Czarnecki, W. M., Osindero, S., Vinyals, O., Graves, A., Silver, D., and Kavukcuoglu, K. Decoupled neural interfaces using synthetic gradients. In *International conference on machine learning*, pp. 1627–1635. PMLR, 2017.
- Journé, A., Rodriguez, H. G., Guo, Q., and Moraitis, T. Hebbian deep learning without feedback. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=8gd4M-_Rjl.
- Krizhevsky, A. Learning multiple layers of features from tiny images. 2009. URL <https://api.semanticscholar.org/CorpusID:18268744>.
- Le, Y. and Yang, X. S. Tiny imagenet visual recognition challenge. 2015. URL <https://api.semanticscholar.org/CorpusID:16664790>.
- Lee, D.-H., Zhang, S., Fischer, A., and Bengio, Y. Difference target propagation. In *Joint european conference on machine learning and knowledge discovery in databases*, pp. 498–515. Springer, 2015.
- Li, Q., Teh, Y. W., and Pascanu, R. Noprop: Training neural networks without full back-propagation or full forward-propagation, 2025. URL <https://arxiv.org/abs/2503.24322>.
- Liang, P., Zhang, W., Wang, J., and Guo, Y. Neighbor self-knowledge distillation. *Information Sciences*, 654: 119859, 2024.
- Lillicrap, T. P., Cownden, D., Tweed, D. B., and Akerman, C. J. Random feedback weights support learning in deep neural networks, 2014. URL <https://arxiv.org/abs/1411.0247>.
- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=PqvMRDCJT9t>.
- Liu, J., Zhang, C., Guo, J., Zhang, Y., Que, H., Deng, K., Liu, J., Zhang, G., Wu, Y., Liu, C., et al. Ddk: Distilling domain knowledge for efficient large language models. *Advances in Neural Information Processing Systems*, 37: 98297–98319, 2024.
- Ma, W.-D. K., Lewis, J. P., and Kleijn, W. B. The hsc bottleneck: Deep learning without back-propagation, 2019. URL <https://arxiv.org/abs/1908.01580>.
- Nøkland, A. Direct feedback alignment provides learning in deep neural networks. *Advances in neural information processing systems*, 29, 2016.
- Nøkland, A. and Eidnes, L. H. Training neural networks with local error signals. In *Proceedings of International Conference on Machine Learning*, pp. 4839–4850. PMLR, 2019.
- Pei, H., Wei, B., Chang, K. C.-C., Lei, Y., and Yang, B. Geom-gcn: Geometric graph convolutional networks. *arXiv preprint arXiv:2002.05287*, 2020.
- Qiu, X., Gan, Y., Hayes, C. F., Liang, Q., Meyerson, E., Hodjat, B., and Miikkulainen, R. Evolution strategies at scale: LLM fine-tuning beyond reinforcement learning. *arXiv preprint arXiv:2509.24372*, 2025.
- Ren, L. and Li, Q. Unifying back-propagation and forward-forward algorithms through model predictive control, 2024. URL <https://arxiv.org/abs/2409.19561>.
- Rivaud, S., Fournier, L., Pumir, T., Belilovsky, E., Eickenberg, M., and Oyallon, E. PETRA: Parallel end-to-end training with reversible architectures. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=0fhzSFsGUT>.
- Scellier, B. and Bengio, Y. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in computational neuroscience*, 11:24, 2017.

- Simonyan, K. and Zisserman, A. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- Sun, L., Zhang, Y., He, W., Wen, J., Shen, L., and Xie, W. Deeperforward: Enhanced forward-forward training for deeper and better performance. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=kOYnXVQCtA>.
- Whittington, J. C. and Bogacz, R. An approximation of the error backpropagation algorithm in a predictive coding network with local hebbian synaptic plasticity. *Neural computation*, 29(5):1229–1262, 2017.
- Wu, D. Y., Lin, D., Chen, V., and Chen, H.-H. Associated learning: an alternative to end-to-end backpropagation that works on cnn, rnn, and transformer. In *International Conference on Learning Representations*, 2021.
- Xu, J., Li, J., and Zhang, J. Scalable model merging with progressive layer-wise distillation. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=xX8NJShgny>.
- Yang, Z., Cohen, W., and Salakhudinov, R. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, pp. 40–48. PMLR, 2016.
- Yin, B. and Corradi, F. Stochastic layer-wise learning: Scalable and efficient alternative to backpropagation. *arXiv preprint arXiv:2505.05181*, 2025.

Appendix

• A Proofs for Theoretical Analysis	12
– A.1 Proof of Proposition 4.2 (Geometric Error Decay)	12
– A.2 Proof of Proposition 4.3 (Global Performance Bound)	14
• B Extended Theoretical Analysis	14
– B.1 Convergence Analysis under Non-Stationary Targets	14
– B.2 Approximation Error Bounds under Restricted Function Class	15
– B.3 Connection to Functional Gradient Descent and Boosting	16
– B.4 Theoretical Analysis under Imperfect Optimization (ϵ -Robustness)	17
– B.5 Analysis of Stale Teacher Beliefs	18
• C Detailed Experimental Information	19
– C.1 Experimental Setup Details (Datasets, Architectures, Training Protocol)	19
– C.2 Versatility and Generalization (GNNs, Diffusion Models)	21
• D Supplemental Experimental Results	23
– D.1 Ablation Studies (α Sensitivity, Update Strategy)	23
– D.2 Additional Visualizations (Feature Space, Belief Evolution)	23
– D.3 Robustness to Label Noise	24
– D.4 Computational Performance Analysis Methodology	24
• E Architectural Adaptability and Limitations	25
– E.1 Design Philosophy: The ResNet-DMoL Adapter	25
– E.2 Architectural Adaptation: The ResNet-DMoL-RA	25

A. Proofs for Theoretical Analysis

In this appendix, we provide the detailed proofs for the propositions presented in Section 4. Throughout the proofs, we denote a discrete probability distribution over $\mathcal{Y} = \{1, \dots, m\}$ as a vector $p \in \Delta^{m-1}$, where Δ^{m-1} is the $(m-1)$ -simplex. All KL divergences assume distributions have full support, which can be ensured in practice via label smoothing.

A.1. Proof of Proposition 4.2 (Geometric Error Decay)

The proof relies on first establishing the closed-form solution for the local optimization problem.

Lemma A.1 (Closed-Form Local Minimizer). *Given fixed distributions p_{i-1} and p_y with full support, the local objective functional $\mathcal{L}_i(p) = \alpha D_{KL}(p||p_y) + (1 - \alpha) D_{KL}(p||p_{i-1})$ is strictly convex. Its unique minimizer, $p_i^* \triangleq \arg \min_p \mathcal{L}_i(p)$, is given by the geometric interpolation:*

$$p_i^*(k) = \frac{1}{Z_i} p_{i-1}(k)^{1-\alpha} p_y(k)^\alpha, \quad \text{where } Z_i = \sum_{j=1}^m p_{i-1}(j)^{1-\alpha} p_y(j)^\alpha. \quad (6)$$

Proof of Lemma A.1. The KL divergence is strictly convex, and a non-negative weighted sum of strictly convex functions preserves strict convexity. Thus, $\mathcal{L}_i(p)$ has a unique global minimizer. We solve for it using Lagrange multipliers with the constraint $\sum_k p(k) = 1$. The Lagrangian is:

$$\mathcal{J}(p, \lambda) = \alpha \sum_k p(k) \log \frac{p(k)}{p_y(k)} + (1 - \alpha) \sum_k p(k) \log \frac{p(k)}{\mathbf{p}_{i-1}(k)} + \lambda \left(1 - \sum_k p(k) \right). \quad (7)$$

Setting the partial derivative w.r.t. $p(k)$ to zero:

$$\frac{\partial \mathcal{J}}{\partial p(k)} = \alpha(\log p(k) + 1 - \log p_y(k)) + (1 - \alpha)(\log p(k) + 1 - \log \mathbf{p}_{i-1}(k)) - \lambda = 0 \quad (8)$$

$$\implies \log p(k) + 1 - \alpha \log p_y(k) - (1 - \alpha) \log \mathbf{p}_{i-1}(k) = \lambda \quad (9)$$

$$\implies \log p(k) = (\lambda - 1) + \log (\mathbf{p}_{i-1}(k)^{1-\alpha} p_y(k)^\alpha). \quad (10)$$

Exponentiating both sides yields $p(k) \propto \mathbf{p}_{i-1}(k)^{1-\alpha} p_y(k)^\alpha$. The normalization constant Z_i follows directly from the constraint $\sum_k p(k) = 1$. $\square$

Proof of Proposition 4.2. We proceed by induction on the depth i .

Base Case ($i = 1$): With p_0 as the uniform prior, Lemma A.1 gives:

$$p_1(k) \propto p_0(k)^{1-\alpha} p_y(k)^\alpha.$$

This matches the proposition's formula $\mathbf{p}_i(k) \propto p_0(k)^{(1-\alpha)^i} p_y(k)^{1-(1-\alpha)^i}$ for $i = 1$, noting that $1 - (1 - \alpha)^1 = \alpha$.

Inductive Step: Assume the formula holds for $i - 1$:

$$\mathbf{p}_{i-1}(k) \propto p_0(k)^{(1-\alpha)^{i-1}} p_y(k)^{1-(1-\alpha)^{i-1}}.$$

For module i , substituting $\mathbf{p}_{i-1}(k)$ into the minimizer form:

$$\mathbf{p}_i(k) \propto \mathbf{p}_{i-1}(k)^{1-\alpha} p_y(k)^\alpha \quad (11)$$

$$\propto \left(p_0(k)^{(1-\alpha)^{i-1}} p_y(k)^{1-(1-\alpha)^{i-1}} \right)^{1-\alpha} p_y(k)^\alpha \quad (12)$$

$$\propto p_0(k)^{(1-\alpha)^{i-1}(1-\alpha)} \cdot p_y(k)^{(1-(1-\alpha)^{i-1})(1-\alpha)} \cdot p_y(k)^\alpha \quad (13)$$

$$\propto p_0(k)^{(1-\alpha)^i} \cdot p_y(k)^{1-\alpha-(1-\alpha)^i+\alpha} \quad (14)$$

$$\propto p_0(k)^{(1-\alpha)^i} p_y(k)^{1-(1-\alpha)^i}. \quad (15)$$

This completes the induction.

Convergence Analysis: Since the consistency weight $\alpha \in (0, 1)$, the term $(1 - \alpha) \in (0, 1)$. As $i \rightarrow \infty$:

- The exponent of the prior, $(1 - \alpha)^i$, decays geometrically to 0.
- The exponent of the target, $1 - (1 - \alpha)^i$, approaches 1.

Thus, $\lim_{i \rightarrow \infty} \mathbf{p}_i(k) \propto p_0(k)^0 p_y(k)^1 = p_y(k)$. The belief converges pointwise to the target distribution with a geometric rate determined by $1 - \alpha$. $\square$

A.2. Proof of Proposition 4.3 (Global Performance Bound)

We are given the local improvement condition: $\mathcal{L}_i(\mathbf{p}_i) \leq \mathcal{L}_i(\mathbf{p}_{i-1})$. Expanding the objective function:

$$\mathcal{L}_i(\mathbf{p}_i) = \alpha D_{\text{KL}}(\mathbf{p}_i \| p_y) + (1 - \alpha) D_{\text{KL}}(\mathbf{p}_i \| \mathbf{p}_{i-1}) \quad (16)$$

$$\mathcal{L}_i(\mathbf{p}_{i-1}) = \alpha D_{\text{KL}}(\mathbf{p}_{i-1} \| p_y) + (1 - \alpha) \underbrace{D_{\text{KL}}(\mathbf{p}_{i-1} \| \mathbf{p}_{i-1})}_0 = \alpha D_{\text{KL}}(\mathbf{p}_{i-1} \| p_y). \quad (17)$$

Substituting these into the inequality $\mathcal{L}_i(\mathbf{p}_i) \leq \mathcal{L}_i(\mathbf{p}_{i-1})$:

$$\alpha D_{\text{KL}}(\mathbf{p}_i \| p_y) + (1 - \alpha) D_{\text{KL}}(\mathbf{p}_i \| \mathbf{p}_{i-1}) \leq \alpha D_{\text{KL}}(\mathbf{p}_{i-1} \| p_y).$$

Since $D_{\text{KL}}(\mathbf{p}_i \| \mathbf{p}_{i-1}) \geq 0$ and $\alpha > 0$, we rearrange to bound the target divergence at step i :

$$\alpha D_{\text{KL}}(\mathbf{p}_i \| p_y) \leq \alpha D_{\text{KL}}(\mathbf{p}_{i-1} \| p_y) - (1 - \alpha) D_{\text{KL}}(\mathbf{p}_i \| \mathbf{p}_{i-1}).$$

Dividing by α yields the single-step descent bound:

$$D_{\text{KL}}(\mathbf{p}_i \| p_y) \leq D_{\text{KL}}(\mathbf{p}_{i-1} \| p_y) - \frac{1 - \alpha}{\alpha} D_{\text{KL}}(\mathbf{p}_i \| \mathbf{p}_{i-1}). \quad (18)$$

Summing this inequality over the cascade from $i = 1$ to L :

$$\sum_{i=1}^L [D_{\text{KL}}(\mathbf{p}_i \| p_y) - D_{\text{KL}}(\mathbf{p}_{i-1} \| p_y)] \leq -\frac{1 - \alpha}{\alpha} \sum_{i=1}^L D_{\text{KL}}(\mathbf{p}_i \| \mathbf{p}_{i-1}). \quad (19)$$

The left-hand side is a telescoping sum that collapses to $D_{\text{KL}}(\mathbf{p}_L \| p_y) - D_{\text{KL}}(\mathbf{p}_0 \| p_y)$. Thus:

$$D_{\text{KL}}(\mathbf{p}_L \| p_y) - D_{\text{KL}}(\mathbf{p}_0 \| p_y) \leq -\frac{1 - \alpha}{\alpha} \sum_{i=1}^L D_{\text{KL}}(\mathbf{p}_i \| \mathbf{p}_{i-1}).$$

Rearranging terms yields the final global bound:

$$D_{\text{KL}}(\mathbf{p}_L \| p_y) \leq D_{\text{KL}}(\mathbf{p}_0 \| p_y) - \frac{1 - \alpha}{\alpha} \sum_{i=1}^L D_{\text{KL}}(\mathbf{p}_i \| \mathbf{p}_{i-1}). \quad (20)$$

□

B. Extended Theoretical Analysis

In this section, we deepen our theoretical analysis to address practical constraints encountered in deep learning. We relax the ideal assumptions made in Section 4 to analyze the convergence of DMoL under non-stationary teacher distributions, restricted function classes, and imperfect optimization steps. Furthermore, we establish a formal equivalence between DMoL and Entropic Mirror Descent, proving that the framework operates as a Gradient Boosting machine in the log-probability space.

B.1. Convergence Analysis under Non-Stationary Targets

In the standard DMoL algorithm, the teacher belief $\mathbf{p}_{i-1}$ for module i is generated by module $i - 1$, whose parameters θ_{i-1} are simultaneously being updated. This creates a non-stationary optimization landscape where the target is moving. Here, we prove that this hierarchical coupled dynamic is contractive and stable.

We define the *Ideal Operator* $T : \Delta^{m-1} \times \Delta^{m-1} \rightarrow \Delta^{m-1}$ as the mapping that produces the optimal local minimizer derived in Lemma A.1:

$$T(p, p_y) \triangleq \frac{1}{Z} p^{1-\alpha} p_y^\alpha. \quad (21)$$

Lemma B.1 (Contraction of the Ideal Operator). *The operator $T(\cdot, p_y)$ is a contraction mapping with respect to the logarithmic metric $d_{\log}(p, q) = \|\log p - \log q\|_2$. Specifically, for any beliefs $p, q \in \Delta^{m-1}$:*

$$d_{\log}(T(p, p_y), T(q, p_y)) = (1 - \alpha)d_{\log}(p, q). \quad (22)$$

Proof. In the logarithmic domain (logit space modulo normalization), the geometric interpolation becomes a convex combination:

$$\log T(p, p_y) = (1 - \alpha) \log p + \alpha \log p_y - \log Z_p \mathbf{1}.$$

Considering the distance between transformed p and q :

$$\begin{aligned} \|\log T(p, p_y) - \log T(q, p_y)\| &= \|((1 - \alpha) \log p + \alpha \log p_y) - ((1 - \alpha) \log q + \alpha \log p_y)\| \\ &= \|(1 - \alpha)(\log p - \log q)\| \\ &= (1 - \alpha)\|\log p - \log q\|. \end{aligned}$$

Since $\alpha \in (0, 1)$, the contraction factor is $(1 - \alpha) < 1$. This implies that any perturbation in the teacher distribution is damped by the DMoL objective before propagating to the student. $\square$

Theorem B.2 (Recursive Stability of DMoL Dynamics). *Let $\epsilon_i^{(t)} = \|\log \mathbf{p}_i^{(t)} - \log \mathbf{p}_i^{(\infty)}\|$ be the deviation of module i 's output at step t from its converged state. Assume the optimization algorithm at step t finds a solution within δ_t of the current ideal target $T(\mathbf{p}_{i-1}^{(t)}, p_y)$. Then, the error dynamics satisfy:*

$$\epsilon_i^{(t)} \leq \delta_t + (1 - \alpha)\epsilon_{i-1}^{(t)}. \quad (23)$$

Proof. By the triangle inequality, the total error at layer i is bounded by the optimization error plus the target shift error:

$$\begin{aligned} \epsilon_i^{(t)} &= \|\log \mathbf{p}_i^{(t)} - \log \mathbf{p}_i^{(\infty)}\| \\ &\leq \underbrace{\|\log \mathbf{p}_i^{(t)} - \log T(\mathbf{p}_{i-1}^{(t)}, p_y)\|}_{\text{Optimization Error } \delta_t} + \underbrace{\|\log T(\mathbf{p}_{i-1}^{(t)}, p_y) - \log T(\mathbf{p}_{i-1}^{(\infty)}, p_y)\|}_{\text{Target Shift}}. \end{aligned}$$

Using Lemma B.1, the second term is bounded by $(1 - \alpha)\|\log \mathbf{p}_{i-1}^{(t)} - \log \mathbf{p}_{i-1}^{(\infty)}\| = (1 - \alpha)\epsilon_{i-1}^{(t)}$. Thus, $\epsilon_i^{(t)} \leq \delta_t + (1 - \alpha)\epsilon_{i-1}^{(t)}$. $\square$

Remark: This theorem formally justifies the stability of DMoL in deep networks. The error from previous layers does not accumulate multiplicatively (as gradients do in BP); instead, it is exponentially damped by the factor $(1 - \alpha)$ at each layer.

B.2. Approximation Error Bounds under Restricted Function Class

In practice, the neural network module f_i belongs to a restricted function class $\mathcal{F}$ (parameterized by θ_i) and may not have sufficient capacity to perfectly express the ideal geometric interpolation $\mathbf{p}_i^*$. We denote the constrained minimizer as $\hat{p}_i = \arg \min_{p \in \mathcal{F}} \mathcal{L}_i(p)$.

We define the *Approximation Residual* ν_i for layer i as the excess loss incurred due to limited capacity:

$$\nu_i \triangleq \mathcal{L}_i(\hat{p}_i) - \mathcal{L}_i(\mathbf{p}_i^*) \geq 0. \quad (24)$$

Theorem B.3 (Robust Performance Bound). *For a network of L modules with approximation residuals $\{\nu_i\}_{i=1}^L$, the global performance is bounded by:*

$$D_{KL}(\hat{p}_L \| p_y) \leq D_{KL}(p_0 \| p_y) - \frac{1 - \alpha}{\alpha} \sum_{i=1}^L D_{KL}(\hat{p}_i \| \hat{p}_{i-1}) + \frac{1}{\alpha} \sum_{i=1}^L \nu_i. \quad (25)$$

Proof. We begin by upper-bounding the value of the loss function at the ideal minimizer $\mathbf{p}_i^*$. Substituting $\mathbf{p}_i^*$ into the partition function definition from Lemma A.1, we have $\mathcal{L}_i(\mathbf{p}_i^*) = -\log Z_i$. By the convexity of D_{KL} divergence (or

equivalently, applying Hölder's inequality in reverse), the loss of the optimal interpolation is always lower than or equal to the loss of the input belief itself:

$$\mathcal{L}_i(\mathbf{p}_i^*) \leq \alpha D_{\text{KL}}(\hat{p}_{i-1} \| p_y) + (1 - \alpha) D_{\text{KL}}(\hat{p}_{i-1} \| \hat{p}_{i-1}) = \alpha D_{\text{KL}}(\hat{p}_{i-1} \| p_y). \quad (26)$$

Now, consider the actual loss achieved by the network $\mathcal{L}_i(\hat{p}_i)$. By definition of the residual ν_i :

$$\begin{aligned} \mathcal{L}_i(\hat{p}_i) &= \alpha D_{\text{KL}}(\hat{p}_i \| p_y) + (1 - \alpha) D_{\text{KL}}(\hat{p}_i \| \hat{p}_{i-1}) \\ &= \mathcal{L}_i(\mathbf{p}_i^*) + \nu_i. \end{aligned}$$

Substituting the bound from Eq. (26) into the RHS:

$$\alpha D_{\text{KL}}(\hat{p}_i \| p_y) + (1 - \alpha) D_{\text{KL}}(\hat{p}_i \| \hat{p}_{i-1}) \leq \alpha D_{\text{KL}}(\hat{p}_{i-1} \| p_y) + \nu_i. \quad (27)$$

Rearranging to isolate the target divergence term:

$$D_{\text{KL}}(\hat{p}_i \| p_y) \leq D_{\text{KL}}(\hat{p}_{i-1} \| p_y) - \frac{1 - \alpha}{\alpha} D_{\text{KL}}(\hat{p}_i \| \hat{p}_{i-1}) + \frac{1}{\alpha} \nu_i. \quad (28)$$

Summing this inequality from $i = 1$ to L :

$$\sum_{i=1}^L (D_{\text{KL}}(\hat{p}_i \| p_y) - D_{\text{KL}}(\hat{p}_{i-1} \| p_y)) \leq \sum_{i=1}^L \left(-\frac{1 - \alpha}{\alpha} D_{\text{KL}}(\hat{p}_i \| \hat{p}_{i-1}) + \frac{1}{\alpha} \nu_i \right). \quad (29)$$

The LHS is a telescoping sum resulting in $D_{\text{KL}}(\hat{p}_L \| p_y) - D_{\text{KL}}(p_0 \| p_y)$. Rearranging terms yields the final bound. $\square$

Remark: This bound decomposes the final error into three intelligible components: the initial ignorance ($D_{\text{KL}}(p_0 \| p_y)$), the cumulative capacity penalty ($\sum \nu_i$), and a crucial negative term representing the *Consistency Gain*. This proves that the local consistency constraint $(1 - \alpha) D_{\text{KL}}(\hat{p}_i \| \hat{p}_{i-1})$ acts as a regularizer that actively reduces the upper bound on the global error.

B.3. Connection to Functional Gradient Descent and Boosting

While DMoL is presented as a belief refinement framework, we provide a rigorous analysis showing it is formally equivalent to **Entropic Mirror Descent** and operates as a **Gradient Boosting Machine** in the log-probability space.

B.3.1. DMoL AS ENTROPIC MIRROR DESCENT

Let $J(p) = D_{\text{KL}}(p_y \| p)$ be the global cross-entropy objective we wish to minimize. The gradient of this objective with respect to the belief p (in the natural gradient sense) corresponds to $-\log p_y$.

Theorem B.4. *The DMoL local update is equivalent to a single step of Mirror Descent on the probability simplex with KL divergence as the Bregman divergence (Entropic Mirror Descent).*

Proof. Recall the general update rule for Mirror Descent with step size η and proximal term D_ψ :

$$p_{\text{new}} = \arg \min_{p \in \Delta} \left(\langle \nabla J(p_{\text{old}}), p \rangle + \frac{1}{\eta} D_\psi(p \| p_{\text{old}}) \right).$$

Let us rearrange the DMoL objective $\mathcal{L}_i(p)$:

$$\begin{aligned} \arg \min_p \mathcal{L}_i(p) &= \arg \min_p \left(\alpha \sum p \log \frac{p}{p_y} + (1 - \alpha) \sum p \log \frac{p}{p_{i-1}} \right) \\ &= \arg \min_p \left(-\alpha \sum p \log p_y + \sum p \log p - (1 - \alpha) \sum p \log p_{i-1} \right). \end{aligned}$$

Grouping terms dependent on $D_{\text{KL}}(p\|\mathbf{p}_{i-1}) = \sum p \log p - \sum p \log \mathbf{p}_{i-1}$:

$$\begin{aligned} &= \arg \min_p \left(-\alpha \sum p \log p_y + \alpha \sum p \log \mathbf{p}_{i-1} + (1 - \alpha) D_{\text{KL}}(p\|\mathbf{p}_{i-1}) \right) \\ &= \arg \min_p \left(\alpha \sum p (\log \mathbf{p}_{i-1} - \log p_y) + (1 - \alpha) D_{\text{KL}}(p\|\mathbf{p}_{i-1}) \right). \end{aligned}$$

Dividing by α and noticing $-\log p_y$ is the gradient, we match the form:

$$\arg \min_p \left(\langle -\log p_y, p \rangle + \frac{1 - \alpha}{\alpha} D_{\text{KL}}(p\|\mathbf{p}_{i-1}) + \text{const} \right).$$

This matches the Mirror Descent update where the gradient direction is provided by the target $-\log p_y$, the current iterate is $\mathbf{p}_{i-1}$, and the effective learning rate is $\eta = \frac{\alpha}{1-\alpha}$. $\square$

B.3.2. DMoL AS LOGIT-SPACE GRADIENT BOOSTING

To analyze the constructive capability of DMoL, we transform the dynamics into the unnormalized log-probability (logit) space. Let $F_i(k) = \log \mathbf{p}_i(k)$.

Theorem B.5. *DMoL performs Gradient Boosting on the log-likelihood function, where each module fits the residual between the target log-likelihood and the current belief.*

Proof. From Lemma A.1, the optimal update is $\mathbf{p}_i^* \propto \mathbf{p}_{i-1}^{1-\alpha} p_y^\alpha$. Taking the logarithm:

$$\begin{aligned} \log \mathbf{p}_i^* &= (1 - \alpha) \log \mathbf{p}_{i-1} + \alpha \log p_y + C \\ F_i^* &= F_{i-1} + \alpha (\log p_y - \log \mathbf{p}_{i-1}) \\ F_i^* &= F_{i-1} + \alpha \underbrace{(F_{\text{target}} - F_{i-1})}_{\text{Log-Likelihood Residual}}. \end{aligned}$$

This is the exact functional form of a Boosting update $F_m = F_{m-1} + \eta \cdot h_m$, where the learning rate (shrinkage) is α , and the “weak learner” (module f_i) is trained to approximate the residual vector $r_i = \log(p_y/\mathbf{p}_{i-1})$. $\square$

Convergence with Weak Learners. Standard Boosting theory guarantees convergence if the weak learner is slightly better than random guessing. In our context, this implies that as long as the module f_i can produce an update ΔF_i that has positive correlation with the residual r_i (i.e., $\langle \Delta F_i, r_i \rangle \geq \gamma \|r_i\|^2$ for some $\gamma > 0$), the global error decays exponentially with depth:

$$D_{\text{KL}}(p_L\|p_y) \leq (1 - \alpha\gamma)^L D_{\text{KL}}(p_0\|p_y).$$

This provides a rigorous theoretical justification for DMoL’s ability to achieve high accuracy in deep networks even if individual modules have limited capacity, as they cooperatively boost the total system performance.

B.4. Theoretical Analysis under Imperfect Optimization (ϵ -Robustness)

Proposition 4.3 in the main paper establishes a monotonic descent guarantee under the ideal condition that each module improves its local objective, i.e., $\mathcal{L}_i(\mathbf{p}_i) \leq \mathcal{L}_i(\mathbf{p}_{i-1})$. In practice, due to stochastic gradient descent and finite model capacity, modules may only find an approximate minimum. This section extends our analysis to this more realistic scenario.

We introduce the concept of an ϵ -improvement step, where each module f_i produces an output belief $\mathbf{p}_i$ that satisfies:

$$\mathcal{L}_i(\mathbf{p}_i) \leq \mathcal{L}_i(\mathbf{p}_{i-1}) + \epsilon_i, \tag{30}$$

where $\epsilon_i \geq 0$ is the optimization error for module i . This error can arise from sources like gradient noise or terminating the optimization process early.

Proposition B.6 (Monotonic Descent with Bounded Error). *Suppose that each module’s output $\mathbf{p}_i$ satisfies the ϵ -improvement condition $\mathcal{L}_i(\mathbf{p}_i) \leq \mathcal{L}_i(\mathbf{p}_{i-1}) + \epsilon_i$. Then, the KL divergence of the final belief p_L to the target distribution p_y is bounded as follows:*

$$D_{KL}(p_L \| p_y) \leq D_{KL}(p_0 \| p_y) - \frac{1-\alpha}{\alpha} \sum_{i=1}^L D_{KL}(\mathbf{p}_i \| \mathbf{p}_{i-1}) + \frac{1}{\alpha} \sum_{i=1}^L \epsilon_i. \quad (31)$$

This implies a simpler, more direct upper bound on the final performance degradation:

$$D_{KL}(p_L \| p_y) \leq D_{KL}(p_0 \| p_y) + \frac{1}{\alpha} \sum_{i=1}^L \epsilon_i. \quad (32)$$

Proof. We start from the proof of Proposition 4.3 in Appendix A.2. The ϵ -improvement condition is:

$$\alpha D_{KL}(\mathbf{p}_i \| p_y) + (1-\alpha) D_{KL}(\mathbf{p}_i \| \mathbf{p}_{i-1}) \leq \alpha D_{KL}(\mathbf{p}_{i-1} \| p_y) + \epsilon_i.$$

Rearranging to isolate $D_{KL}(\mathbf{p}_i \| p_y)$:

$$\alpha D_{KL}(\mathbf{p}_i \| p_y) \leq \alpha D_{KL}(\mathbf{p}_{i-1} \| p_y) - (1-\alpha) D_{KL}(\mathbf{p}_i \| \mathbf{p}_{i-1}) + \epsilon_i.$$

Dividing by $\alpha > 0$:

$$D_{KL}(\mathbf{p}_i \| p_y) \leq D_{KL}(\mathbf{p}_{i-1} \| p_y) - \frac{1-\alpha}{\alpha} D_{KL}(\mathbf{p}_i \| \mathbf{p}_{i-1}) + \frac{\epsilon_i}{\alpha}.$$

Summing this inequality from $i = 1$ to L yields a telescoping series on the left-hand side:

$$\sum_{i=1}^L (D_{KL}(\mathbf{p}_i \| p_y) - D_{KL}(\mathbf{p}_{i-1} \| p_y)) \leq \sum_{i=1}^L \left(-\frac{1-\alpha}{\alpha} D_{KL}(\mathbf{p}_i \| \mathbf{p}_{i-1}) + \frac{\epsilon_i}{\alpha} \right).$$

$$D_{KL}(p_L \| p_y) - D_{KL}(p_0 \| p_y) \leq -\frac{1-\alpha}{\alpha} \sum_{i=1}^L D_{KL}(\mathbf{p}_i \| \mathbf{p}_{i-1}) + \frac{1}{\alpha} \sum_{i=1}^L \epsilon_i.$$

Rearranging gives the first result. Since $D_{KL}(\mathbf{p}_i \| \mathbf{p}_{i-1}) \geq 0$, we can drop the negative term to obtain the simpler upper bound:

$$D_{KL}(p_L \| p_y) \leq D_{KL}(p_0 \| p_y) + \frac{1}{\alpha} \sum_{i=1}^L \epsilon_i.$$

This completes the proof. $\square$

Interpretation: Proposition B.6 provides a crucial robustness guarantee. It shows that the final prediction error is bounded by the initial error (from a uniform belief) plus the accumulated optimization errors from all modules, scaled by $1/\alpha$. As long as the local optimization is reasonably effective (i.e., ϵ_i are small), the overall learning process remains stable and does not diverge. This formalizes the intuition that DMoL is resilient to the imperfections inherent in practical training.

B.5. Analysis of Stale Teacher Beliefs

The two-phase design of DMoL (Algorithm 1) involves generating teacher beliefs $\mathbf{p}_{i-1}^{\text{teacher}}$ using model parameters $\theta^{(t)}$ and then using these fixed teachers to compute gradients for updating to $\theta^{(t+1)}$. In asynchronous or parallel settings, these teachers might become “stale”, meaning they were generated with slightly older parameters $\theta^{(t-k)}$ while the update happens on $\theta^{(t)}$. We analyze the impact of this staleness on the gradient computation.

Let θ denote the parameters of a module f_i , and let $\mathbf{p}_{i-1} = f_{i-1}(\dots; \theta_{i-1}^{\text{old}})$ be the stale teacher belief generated with old parameters. The “fresh” teacher would be $\mathbf{p}'_{i-1} = f_{i-1}(\dots; \theta_{i-1}^{\text{new}})$. The local loss for module i is computed using the stale teacher:

$$\mathcal{L}_i(\theta_i) = \alpha D_{KL}(f_i(\dots) \| p_y) + (1-\alpha) D_{KL}(f_i(\dots) \| \text{sg}(\mathbf{p}_{i-1})).$$

The true gradient should use $\mathbf{p}'_{i-1}$. Let $g_i = \nabla_{\theta_i} \mathcal{L}_i$ be the computed gradient and g'_i be the true gradient. The error is in the consistency term’s gradient.

Proposition B.7 (Gradient Error Bound from Staleness). *Assume the module function f_i is L_f -Lipschitz continuous with respect to its parameters, and the gradient of the KL divergence with respect to its second argument is L_{KL} -Lipschitz continuous in the relevant domain. The error in the computed gradient for module i is bounded by:*

$$\|g_i - g'_i\| \leq C \cdot \|\theta_{i-1}^{\text{new}} - \theta_{i-1}^{\text{old}}\|, \quad (33)$$

for some constant C that depends on α , L_f , and L_{KL} .

Proof Sketch. The difference between the gradients is:

$$\|g_i - g'_i\| = (1 - \alpha) \|\nabla_{\theta_i} D_{\text{KL}}(f_i(\dots) \| \mathbf{p}_{i-1}) - \nabla_{\theta_i} D_{\text{KL}}(f_i(\dots) \| \mathbf{p}'_{i-1})\|.$$

Using the chain rule and Lipschitz continuity assumptions, this difference can be bounded by the difference in the teacher beliefs, $\|\mathbf{p}_{i-1} - \mathbf{p}'_{i-1}\|$. This belief difference is, in turn, bounded by the Lipschitz continuity of the predecessor module f_{i-1} :

$$\|\mathbf{p}_{i-1} - \mathbf{p}'_{i-1}\| = \|f_{i-1}(\dots; \theta_{i-1}^{\text{old}}) - f_{i-1}(\dots; \theta_{i-1}^{\text{new}})\| \leq L_f \cdot \|\theta_{i-1}^{\text{old}} - \theta_{i-1}^{\text{new}}\|.$$

Combining these bounds yields the result. $\square$

Interpretation: The gradient error is directly proportional to the magnitude of parameter change between the time of teacher generation and its use. In standard training with small learning rates, $\|\theta^{\text{new}} - \theta^{\text{old}}\|$ is small, ensuring the gradient error is minimal. This analysis provides confidence that the two-phase approach is stable and the use of cached teachers is a valid and robust approximation.

C. Detailed Experimental Information

This appendix provides comprehensive details regarding the experimental setup, supplemental results, and analysis methodologies discussed in the main paper. Our goal is to ensure full reproducibility and provide deeper insights into our findings.

C.1. Experimental Setup Details

C.1.1. DATASETS AND PREPROCESSING

We used standard benchmarks across vision and graph domains.

Vision Datasets. Standard data augmentation techniques were applied during training.

- **CIFAR-10 & CIFAR-100:** For both datasets (Krizhevsky, 2009), we used the standard training set of 50,000 images and a test set of 10,000 images. The images are 32x32 pixels. For data augmentation, we applied random horizontal flips and random 32x32 crops from images zero-padded to 40x40.
- **Tiny-ImageNet:** This dataset (Le & Yang, 2015) contains 200 classes from ImageNet, with 100,000 training images, 10,000 validation images, and 10,000 test images. Images are downsampled to 64x64 resolution. For augmentation, we used random horizontal flips and random 64x64 crops from images zero-padded to 72x72.

All images were normalized using the per-channel mean and standard deviation computed from their respective training sets.

Graph Datasets. For the versatility experiments on Deep GNNs, we evaluated on three standard citation/co-authorship datasets: **Cora**, **PubMed**, and **Actor**. These datasets are standard benchmarks for node classification tasks and are particularly prone to over-smoothing in deep architectures.

C.1.2. ARCHITECTURES

SimpleCNN Architecture. Our primary testbed, the SimpleCNN, was designed with a deep stack of processing modules to rigorously test optimization algorithms. It separates perceptual feature extraction from sequential belief refinement. The architecture is detailed in Table 4.

Table 4. The SimpleCNN architecture. The network consists of a shared feature extractor followed by L identical processing modules.

Component	Layer Type	Output Shape / Dims
Shared Feature Extractor (c)		
	Conv(3, 32, k=3), ReLU	(B, 32, H, W)
	Conv(32, 64, k=3), ReLU, MaxPool(2)	(B, 64, H/2, W/2)
	Conv(64, 128, k=3), ReLU	(B, 128, H/2, W/2)
	Conv(128, 128, k=3), ReLU, MaxPool(2)	(B, 128, H/4, W/4)
	AdaptiveAvgPool(1), Flatten	(B, 128)
	Linear(128, 128)	(B, 128) $\rightarrow z$
Processing Module (f_i), repeated L times		
	Input Concatenation	(B, 128 + num_classes) $\rightarrow \text{cat}(\mathbf{p}_{i-1}, z)$
	Linear, ReLU	(B, 256)
	Linear	(B, num_classes) $\rightarrow$ Logits for $\mathbf{p}_i$

ViT-Tiny Architecture. For ImageNet experiments, we employ a Vision Transformer (ViT-Tiny) architecture. The model processes images at 224×224 resolution. Unlike the sequential accumulation in ResNets, the ViT maintains a constant embedding dimension across layers. To adapt this for DMoL, we attach a local linear projection head to the [CLS] token output of each Transformer block. The architecture specifications are detailed in Table 5.

 Table 5. The ViT-Tiny architecture used for ImageNet. Each Transformer block acts as a DMoL module (f_i) by attaching a local head to the [CLS] token.

Component	Configuration	Details
Input Processing		
	Patch Embedding	Patch Size 16×16 (or 4 for CIFAR), Dim 192
	Positional Embedding	Learnable, Dim 192
	[CLS] Token	Learnable, Dim 192
Transformer Block (f_i), repeated $L = 12$ times		
	Attention Mechanism	Heads 3, Dim 192, QKV Bias
	MLP	Ratio 4.0, GELU Activation
DMoL Local Head (h_i), attached to every block		
	Input	[CLS] token feature $z_i \in \mathbb{R}^{192}$
	Projection	Linear(192 $\rightarrow$ num_classes)

Standard Architectures. For the architectural generality experiments, we used standard, off-the-shelf implementations of VGG-11, ResNet-18, and a ViT-Tiny model adapted for CIFAR-scale images.

GNN Architecture. We constructed a deep **8-layer GCN** backbone. Each layer constitutes a DMoL module equipped with a local linear readout head.

C.1.3. BASELINE IMPLEMENTATION DETAILS

To ensure a rigorous comparison, we implemented all baselines following their official protocols and best practices for deep architectures.

- **Backpropagation (BP):** Standard end-to-end training using SGD with Nesterov momentum (0.9) and weight decay (5×10^{-4}). We employ a cosine annealing learning rate schedule to ensure optimal convergence.
- **Feedback Alignment (FA):** We utilize the `FALinear` module which replaces the weight transpose W^T in the backward pass with a fixed random matrix B . Crucially, to support training in deeper networks, we initialize B using

the Kaiming Uniform scheme (He et al., 2015) to maintain gradient variance magnitude. We use the Adam optimizer to handle the noisy error signals characteristic of FA.

- **NoProp:** Implemented following Li et al. (2025). The network consists of a shared feature extractor and parallel MLP heads. We employ the target smoothing strategy, where layer-wise targets are generated by mixing the one-hot label with Gaussian noise, controlled by a decay schedule (α_t) to stabilize the layer-wise MSE objectives.
- **Forward-Forward (FF):** Following Hinton (2022), we construct “positive” and “negative” data by overlaying correct/incorrect class labels onto input images using a dedicated `DatasetWrapper`. The network is trained layer-by-layer using a local goodness objective (maximizing squared activity for positive data above a threshold $\theta = 2.0$). A final linear classifier is trained on the concatenated features of all frozen layers.
- **HSIC Bottleneck Learning:** We implement the Greedy Layer-wise Training protocol. Each module is trained to simultaneously minimize the cross-entropy loss of a local auxiliary classifier and maximize the Hilbert-Schmidt Independence Criterion (HSIC) between its output and the input features. We use an RBF kernel with an adaptive bandwidth (σ) set to the batch-wise median distance to ensure numerical stability.
- **Decoupled Greedy Learning (DGL):** Adapted from Belilovsky et al. (2020), this method trains modules sequentially. Crucially, the input to module $i + 1$ is the *detached* output of module i , strictly preventing backward gradient flow. We use a hybrid optimization strategy: SGD for the feature extractor and Adam for subsequent greedy modules to accelerate local convergence.

C.1.4. TRAINING PROTOCOL

To ensure fair comparisons, all methods were trained using the same optimization protocol unless otherwise specified.

- **Optimizer:** AdamW for ViT experiments; SGD with Momentum for CNNs; Adam for specific baselines (FA, DGL parts) where noted.
- **Learning Rate:** Initial rate of 1×10^{-3} for Adam/AdamW, or 0.1 for SGD.
- **Weight Decay:** 5×10^{-4} for all models.
- **Learning Rate Schedule:** A cosine annealing schedule over the course of training.
- **Batch Size:** 128 for all datasets.
- **Epochs:** 100 epochs for benchmark comparisons.
- **Loss Function:** Cross-entropy with label smoothing ($\epsilon = 0.1$) for all applicable methods.
- **DMoL Hyperparameter α :** For Convolutional Neural Networks, which benefit from strong inductive biases, we observe that a fixed balancing coefficient of $\alpha = 0.5$ yields optimal stability.

However, for **Vision Transformers (ViT)**, we employ a **Dynamic α Schedule**. ViTs lack inherent inductive biases (e.g., locality, translation invariance) and are prone to optimization instability in early phases. To address this, we initialize $\alpha = 0.9$ during the warm-up phase. This high value prioritizes *Task Alignment* (direct supervision), forcing all layers to independently learn meaningful representations from the ground truth, preventing the propagation of uninformative priors from shallow layers. As training progresses, we linearly decay α to 0.5. This gradual transition shifts the focus towards *Consistency*, enabling the deep layers to leverage the learned features for cooperative refinement rather than independent prediction.

C.1.5. COMPUTING INFRASTRUCTURE

All experiments were conducted on a server equipped with NVIDIA A100 GPUs using PyTorch.

C.2. Versatility and Generalization to Other Domains

While the main text focuses on computer vision, the principles of **Synergistic Information Distillation (DMoL)** are architecture-agnostic. This section details the experimental configurations and theoretical underpinnings for applying DMoL to **Deep Graph Neural Networks (GNNs)** and **Generative Diffusion Models**.

C.2.1. DEEP GRAPH LEARNING: OVERCOMING OVER-SMOOTHING

The Depth Dilemma in GNNs. Standard GNNs (e.g., GCN, GAT) operate via recursive neighborhood aggregation, which effectively performs Laplacian smoothing on the graph signals. As the network depth L increases, node representations tend to converge to a stationary distribution that depends only on the graph structure, losing input feature information. This phenomenon, known as **over-smoothing**, renders deep node features indistinguishable across classes. Simultaneously, the vanishing gradient problem in End-to-End BP prevents supervision signals from effectively optimizing the lower layers.

Experimental Setup. We evaluated DMoL against a BP baseline on an 8-layer GCN.

- **Baseline (BP):** An 8-layer GCN trained end-to-end with Cross-Entropy loss.
- **Ours (DMoL):** An 8-layer GCN where each layer constitutes a DMoL module equipped with a local linear readout head. The training follows the decoupled protocol described in Section 3.

C.2.2. GENERATIVE DIFFUSION MODELS: DiT-DMoL

The Standard Diffusion Objective. Consider a diffusion model trained to predict the added noise $\epsilon \sim \mathcal{N}(0, I)$ from a noisy input x_t . The standard objective is $\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \mathcal{F}_\theta(x_t, t)\|^2]$, where $\mathcal{F}_\theta$ is a deep neural network (e.g., DiT) composed of L layers: $\mathcal{F}_\theta = f_L \circ f_{L-1} \circ \dots \circ f_1$.

The DiT-DMoL Architecture. To apply DMoL, we modify the DiT architecture by attaching a lightweight **readout head** $h_i(\cdot)$ to the output of each transformer block f_i . Let z_i denote the latent representation output by layer i . The local “belief” at layer i , denoted as $\hat{\epsilon}_i$, represents the noise estimate at that depth:

$$z_i = f_i(z_{i-1}, t), \quad \hat{\epsilon}_i = h_i(z_i). \quad (34)$$

Unlike standard training where only $\hat{\epsilon}_L$ is supervised, DMoL imposes a local objective $\mathcal{L}_i$ on every module.

The Local Diffusion Objective. We adapt the DMoL functional to the regression domain of diffusion. For module i , the loss is defined as a convex combination of a **Distillation Term** (fitting the ground truth noise) and a **Consistency Term** (maintaining fidelity to the previous layer’s estimate):

$$\mathcal{L}_i(\theta_i, \theta_{h_i}) = \underbrace{\alpha \|\epsilon - \hat{\epsilon}_i\|^2}_{\text{Denoising}} + (1 - \alpha) \underbrace{\|\text{sg}(\hat{\epsilon}_{i-1}) - \hat{\epsilon}_i\|^2}_{\text{Stability}}, \quad (35)$$

where $\text{sg}(\cdot)$ is the stop-gradient operator. The consistency target $\hat{\epsilon}_{i-1}$ acts as a “teacher” generated by the previous layer.

Theoretical Justification: Iterative Variance Reduction.

Proposition C.1 (Iterative Variance Reduction). *Let $\mathcal{E}_i = \|\epsilon - \hat{\epsilon}_i\|^2$ be the error at layer i . Assuming the module f_i has sufficient capacity to model the residual error of layer $i - 1$, minimizing $\mathcal{L}_i$ guarantees that the estimator variance is non-increasing with depth, i.e., $\mathbb{E}[\mathcal{E}_i] \leq \mathbb{E}[\mathcal{E}_{i-1}]$.*

Proof. Consider the optimal estimator for layer i that minimizes $\mathcal{L}_i$. Taking the functional derivative with respect to $\hat{\epsilon}_i$ and setting to zero, we find the optimal belief $\hat{\epsilon}_i^*$ is a weighted average of the ground truth and the prior belief:

$$\hat{\epsilon}_i^* = \alpha \epsilon + (1 - \alpha) \hat{\epsilon}_{i-1}. \quad (36)$$

However, the network cannot access ϵ directly during inference; it must learn to approximate this optimum. Let the module f_i learn a residual function r_i such that $\hat{\epsilon}_i = \hat{\epsilon}_{i-1} + r_i(z_{i-1})$. The consistency term $\|\text{sg}(\hat{\epsilon}_{i-1}) - \hat{\epsilon}_i\|^2$ simplifies to $\|r_i\|^2$, acting as an L_2 regularizer on the update magnitude.

The optimization problem for layer i effectively becomes finding a refinement r_i that moves $\hat{\epsilon}_{i-1}$ closer to ϵ . Since $\hat{\epsilon}_{i-1}$ is fixed (via stop-gradient), layer i effectively solves:

$$\min_{\theta_i} \|(\epsilon - \hat{\epsilon}_{i-1}) - r_i(z_{i-1})\|^2. \quad (37)$$

This is equivalent to fitting the **residual noise** that the previous layer failed to capture. As long as there exists any correlation between the features z_{i-1} and the residual error $(\epsilon - \hat{\epsilon}_{i-1})$, gradient descent will find an r_i that reduces the total error variance. Thus, $\mathbb{E}[\mathcal{E}_i] < \mathbb{E}[\mathcal{E}_{i-1}]$. $\square$

Why Lower MSE? While standard BP theoretically allows layer L to minimize the global error, it suffers from the “conflicting gradient” problem in deep denoising networks. The gradient $\nabla_{\theta_1} \mathcal{L}_{BP}$ must traverse L layers, each attempting to capture different frequency components of the noise (e.g., Layer 1 captures low-frequency structure, Layer L captures high-frequency texture).

In BP, the single global error signal forces all layers to compromise, often leading to a stationary point where high-frequency details are averaged out (blurriness). In contrast, DMoL enforces **spectral decomposition via structural regularization**:

1. **Early Layers** (constrained by $\hat{\epsilon}_0 \approx 0$) minimize $\mathcal{L}_1$ by capturing the dominant low-frequency components of ϵ (Global Structure).
2. **Later Layers**, receiving $\hat{\epsilon}_{i-1}$ as a stable anchor, optimize $\mathcal{L}_i$ by fitting only the remaining high-frequency residuals.

This mechanism acts as an implicit **curriculum learning**. The consistency term prevents deep layers from destroying the structural information learned by shallow layers, while the distillation term forces them to refine the details. Consequently, the final estimator $\hat{\epsilon}_L$ is a composition of L specialized experts, leading to a significantly lower final MSE and sharper image generation, even if the convergence speed (rate of loss decrease) is governed by the optimizer and remains similar to BP.

D. Supplemental Experimental Results

D.1. Ablation Studies

We investigate the sensitivity of DMoL to the consistency hyperparameter α and the efficacy of our shared feature extractor update strategy.

Impact of Consistency Weight α . Figure 8 illustrates the test accuracy on CIFAR-10 across varying α values. DMoL exhibits high robustness, maintaining competitive performance for $\alpha \in [0.2, 0.8]$. Performance peaks at $\alpha = 0.5$, which balances the task supervision with the structural regularization from the predecessor belief.

Feature Extractor Update Strategy. A key design choice in DMoL is allowing gradients from all local modules to update the shared feature extractor $c(\cdot)$. Table 6 compares this “all-layer” strategy against two variants: (1) *Final Layer Only*, where only the last module updates the extractor (similar to standard BP but without intermediate updates); and (2) *Frozen*, where the extractor is randomly initialized and fixed. The significant performance drop in the variants confirms that the aggregated gradient signal from all cooperative modules is essential for learning high-quality shared representations.

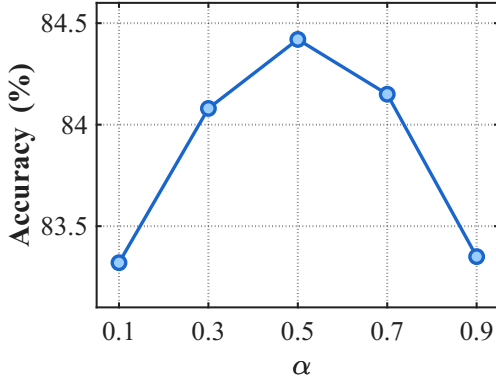


Figure 8. **Sensitivity to α .** Accuracy on CIFAR-10 remains robust across a wide range of α , peaking at 0.5.

Table 6. **Ablation on Update Strategy.** Aggregating gradients from all modules (Default) allows the shared extractor to learn richer representations compared to freezing or single-source updates.

Strategy	CIFAR-10	CIFAR-100
DMoL (Frozen)	82.90%	49.80%
DMoL (Final Layer)	83.85%	52.15%
DMoL (Default)	84.42%	54.05%

D.2. Additional Visualizations

Shared Feature Space Visualization. To verify that the locally supervised extractor learns semantically meaningful features, Figure 9 visualizes the 128-dimensional output of $c(x)$ using t-SNE. Comparing DMoL (left) with BP (right), we observe that DMoL induces a latent space with well-separated class clusters comparable to end-to-end backpropagation.

This validates that the local objectives $\mathcal{L}_i$, despite being decoupled, collectively provide a coherent supervisory signal to the shared backbone.

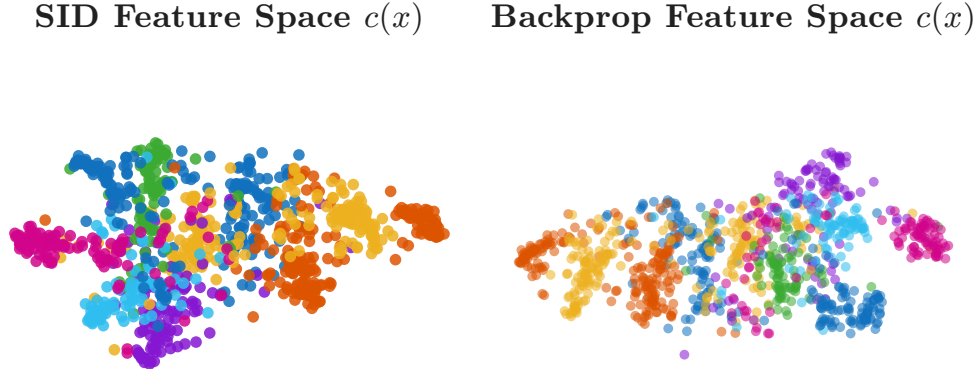


Figure 9. **t-SNE Visualization of Shared Features $c(x)$** . DMoL (Left) learns a discriminative embedding space comparable to End-to-End BP (Right), demonstrating effective representation learning without global gradient propagation.

D.3. Robustness to Label Noise

Experimental Setup. We evaluate robustness by introducing symmetric label noise to the CIFAR-10 training set. Labels for a given percentage of data are randomly flipped to one of the other 9 classes. We compare the test accuracy after 100 epochs.

Analysis. As shown in Table 7, DMoL’s advantage widens as noise increases. At 40% noise, DMoL outperforms BP by **+7.67%**. We attribute this to the local consistency term:

$$(1 - \alpha)D_{\text{KL}}(\mathbf{p}_i \| \text{sg}(\mathbf{p}_{i-1})).$$

When the ground-truth label $\mathbf{p}_y$ is noisy (incorrect), this term acts as a stabilizer, anchoring the update to the consensus of previous layers rather than aggressively overfitting the noisy target. Standard BP, lacking this layer-wise regularization, is more susceptible to memorizing corrupted labels.

Table 7. **Robustness under Symmetric Label Noise.** Test accuracy (%) on CIFAR-10. DMoL demonstrates superior resistance to label corruption compared to BP.

Noise Level	0% (Original)	20%	40%
BP	84.32 $\pm$ 0.45	75.11 $\pm$ 0.62	58.24 $\pm$ 0.88
DMoL (Ours)	84.42 $\pm$ 0.35	78.53 $\pm$ 0.41	65.91 $\pm$ 0.53
<i>Improvement (Δ)</i>	<i>+0.10</i>	<i>+3.42</i>	<i>+7.67</i>

D.4. Computational Performance Analysis Methodology

To rigorously quantify the impact of update locking and validate the architectural advantages of DMoL in a distributed setting, we designed a controlled multi-threaded simulation experiment. Instead of relying on theoretical arithmetic derived from serial execution, we constructed a concurrent execution environment using Python’s threading interface on a high-performance multi-core CPU. In this topology, each network module is assigned to a dedicated thread, effectively modeling a pipeline of independent edge devices or processors. This approach allows us to isolate the algorithmic phenomenon of update locking from the hardware-specific noise of physical interconnects (such as PCIe latency or network bandwidth fluctuations), thereby providing a pure evaluation of the training framework’s structural parallelism capabilities.

The simulation strictly enforces the dependencies inherent to each algorithm. For the standard BP baseline, we implemented a global synchronization lock mechanism that governs the backward phase. A thread managing layer i remains blocked and cannot perform its weight update until it receives the precise gradient signal from layer $i + 1$. This setup faithfully reproduces the “update locking” phenomenon where computational resources remain idle during the sequential propagation of errors. Conversely, for the DMoL framework, we removed these backward dependencies to enable a decoupled producer-consumer

model. Each thread performs its local forward and backward updates immediately upon receiving the necessary activations, requiring synchronization only for the lightweight, gradient-free “teacher” generation pass. This design allows for the formation of an efficient processing pipeline where multiple modules can be active simultaneously.

The primary metric for this evaluation is the normalized training throughput, defined as the number of samples processed per second relative to a single-threaded baseline. We conducted the experiment by progressively increasing the number of active threads (workers) from 1 to L (the network depth), measuring the system’s scaling behavior under both regimes. This methodology provides empirical evidence that while BP’s throughput is fundamentally capped by its sequential critical path, DMoL’s decoupled architecture allows the training throughput to scale robustly with the availability of parallel computational resources, effectively converting additional hardware into training speed.

E. Architectural Adaptability and Limitations

In the main text, we demonstrated DMoL’s effectiveness on architectures naturally suited for belief refinement, such as the VGG-style SimpleCNN where layers act as sequential processors refining a distribution. A critical question remains: Can the DMoL paradigm be generalized to standard feature-centric backbones like ResNet?

In this section, we explore this compatibility by designing a ResNet-DMoL variant. We analyze why simply applying DMoL to feature-extracting architectures leads to a performance trade-off, identifying a fundamental semantic mismatch between “Feature Abstraction” and “Belief Refinement.”

E.1. Design Philosophy: The ResNet-DMoL Adapter

Standard ResNets operate on the principle of hierarchical feature abstraction, where early layers extract low-level patterns (edges, textures) that are semantically distant from the target class. In contrast, DMoL requires every module to output a valid “belief” (a probability distribution over classes).

To bridge this gap without breaking the ResNet structure, we propose an Accumulative Residual Strategy. Instead of forcing intermediate feature maps to be probability distributions, we treat the network as a sequence of residual updates to a global state.

The Accumulative Mechanism. Let h_i be the output feature map of the i -th residual block. In a standard ResNet trained with BP, the features are simply passed forward. In ResNet-DMoL, we define a Cumulative State S_i :

$$S_i = S_{i-1} + h_i \quad (38)$$

Here, S_0 is the initial patch embedding. The state S_i represents the “current best guess” of the image features up to layer i .

Shared Projection to Belief Space. To apply the DMoL objective (which operates on probabilities), we introduce a Shared Classifier Head (f_{head}) that is reused across all layers. The belief at layer i is obtained by projecting the cumulative state:

$$p_i = \text{Softmax}(f_{head}(\text{Norm}(S_i))) \quad (39)$$

This design allows the residual blocks to continue processing features (h_i) while satisfying DMoL’s requirement that every layer produces a prediction (p_i).

Decoupled Training Logic. We maintain DMoL’s core benefit—parallel training—by decoupling the accumulation. During the update of layer i :

1. The previous accumulation S_{i-1} is treated as a fixed, detached “teacher input.”
2. The module generates h_i to optimize p_i (derived from $S_{i-1} + h_i$).
3. The objective forces p_i to approach the ground truth while remaining consistent with the prediction derived from S_{i-1} .

E.2. Architectural Adaptation: The ResNet-DMoL-RA

While DMoL naturally fits architectures designed for sequential processing, applying it to standard feature-centric backbones like ResNet presents a semantic challenge: early convolutional layers extract low-level patterns (e.g., edges) that are distinct

from high-level semantic beliefs. To bridge this gap without disrupting the feature extraction hierarchy, we introduce the Accumulative Residual Adapter (ResNet-DMoL-RA).

Mechanism: Decoupling Features from Beliefs. Instead of forcing the intermediate feature maps x_i to directly represent probability distributions, ResNet-DMoL-RA maintains a global belief state $z_i \in \mathbb{R}^C$ (where C is the class count) that evolves separately from the backbone features. We attach a lightweight *Boosting Head* $B_i(\cdot)$ —composed of Global Average Pooling and a linear projection—to each residual stage. The belief state is updated additively:

$$z_i = \text{sg}(z_{i-1}) + B_i(x_i), \quad (40)$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operator. Here, z_{i-1} acts as a fixed anchor from the previous stage, and the module i learns to predict a correction term $\Delta z_i = B_i(x_i)$ that minimizes the DMoL objective $\mathcal{L}_i$ (Eq. 2).

Theoretical Connection to Gradient Boosting. This architectural modification reveals a profound connection between DMoL and Gradient Boosting Machines (GBMs). In a standard GBM, the model is built by sequentially adding weak learners to fit the negative gradient (residual) of the loss function with respect to the current prediction. Similarly, in ResNet-DMoL-RA, by enforcing the update rule $z_i = z_{i-1} + \Delta z_i$ and minimizing a local objective that penalizes the divergence between z_i and the target y , the Boosting Head B_i is implicitly trained to approximate the *functional gradient* of the classification error in the logit space.

However, a critical distinction remains: while GBMs typically construct learners sequentially using fixed features, ResNet-DMoL-RA trains the feature extractor (x_i) and the boosting head (B_i) simultaneously. The stop-gradient operator ensures that the feature representation at stage i is optimized specifically to correct the semantic deficiencies of stage $i - 1$, effectively realizing a deep, differentiable implementation of functional gradient descent. This design resolves the semantic friction in ResNets, allowing DMoL to scale effectively to complex backbones on benchmarks like Tiny-ImageNet.

F. Detailed Conceptual Comparisons with Prior Arts

To provide a clearer positioning of our contribution, we elaborate on the fundamental conceptual differences between DMoL and several highly relevant prior works discussed in the main text.

Comparison with Stochastic Layer-wise Learning (SLL). While both SLL and DMoL adopt a probabilistic view by mapping activations into distributions, they diverge fundamentally in four aspects: (1) *Projection Mechanism*: SLL relies on fixed random projections, which are task-agnostic and limit the ability to adapt representations to downstream objectives. DMoL utilizes lightweight learnable projection heads, enabling semantically aligned representations. (2) *Structural Robustness*: SLL is designed for fixed-depth execution and does not account for dynamic structural changes. DMoL employs a local belief-driven objective that decouples layer-wise optimization, enabling on-the-fly adaptations (e.g., layer dropping) without performance collapse. (3) *Extensibility*: SLL is primarily demonstrated on standard MLPs/CNNs, whereas DMoL naturally extends to complex domains like Deep GNNs and Diffusion models. (4) *Theoretical Guarantees*: SLL derives from variational ELBO decomposition without guarantees on depth-wise improvement. DMoL operates as an Entropic Mirror Descent with provable exponential convergence and monotonic improvement across layers.

Comparison with Predictive Coding (PC). PC aims to remove gradient transport by training each layer to reconstruct the activity of the preceding layer via local inverse mappings. Consequently, task-relevant supervision is only injected at the output layer and propagated indirectly. This tightly coupled forward-backward interaction across layers leads to strong inter-layer dependencies and structural fragility. In contrast, DMoL provides explicit task-driven supervision at every layer through a strictly forward logit-space objective, completely removing inter-layer backward dependencies and supporting dynamic depth reconfiguration.

Comparison with Difference Target Propagation (DTP). DTP relies on reconstruction via inverse mappings, where task information is introduced at the final layer and propagated backward, leading to backward dependencies and update locking. DMoL adopts a strictly forward teaching paradigm, removing such dependencies and enabling decoupled local updates with better parallelism.

Comparison with Associated Learning (AL). AL relies on heavy feature-space matching using auxiliary autoencoders at intermediate stages, which introduces significant parameter and computational overhead. In contrast, DMoL operates via a minimal belief-space projection (introducing $< 0.1\%$ parameter overhead) and provides a theoretically justified refinement paradigm based on KL divergence.

Comparison with Pipeline Parallelism (e.g., GPipe). GPipe is a system-level pipeline parallelism technique for standard BP. While it improves hardware utilization by overlapping computation, it remains fundamentally bound by global update locking and sequential gradients. It lacks the mathematical decoupling and structural robustness of DMoL.

G. Supplementary Experimental Evidence

This section provides extensive additional empirical evidence to address specific architectural scaling, training dynamics, and efficiency metrics.

G.1. Performance on Heavier Architectures

To verify that DMoL’s decoupled optimization scales effectively to larger and more complex architectures, we evaluated the framework on ViT-Small and DenseNet-121. As shown in Table 8, DMoL achieves highly competitive performance to standard BP. We emphasize that DMoL achieves these results while entirely eliminating global inter-module BP dependencies, leading to more efficient parallel training and structural robustness.

Table 8. Test Accuracy (%) on Heavier Architectures. DMoL scales effectively to deep and complex backbones.

Dataset	Architecture	BP	DMoL (Ours)
CIFAR-100	ViT-Small	81.24 ± 0.15	81.55 ± 0.18
	DenseNet-121	79.64 ± 0.15	77.85 ± 0.28
Tiny-ImageNet	ViT-Small	67.51 ± 0.28	68.04 ± 0.31
	DenseNet-121	64.88 ± 0.22	62.15 ± 0.37

G.2. The Role of Knowledge Distillation: “Delay” vs. “Enhance” Representations

One might hypothesize that the consistency term in DMoL simply constrains layer-wise updates, thereby merely “delaying” accuracy gains to later layers rather than fundamentally enhancing representation learning. We investigated this by tracking the block-wise test accuracy on a 12-block ViT-Small across different Knowledge Distillation (KD) balance configurations (α).

As shown in Table 9 and Table 10, the greedy setting ($\alpha = 1.0$) forces early layers to optimize too sharply (e.g., reaching 68.4% rapidly at Block 4 vs. DMoL’s 56.2%). The gains are indeed “postponed” by DMoL. However, due to the Information Bottleneck, early layers in the greedy model discard complex patterns, causing premature dimensional collapse. Lacking backward error-correction, deeper layers cannot recover this information, causing the greedy model to flatline at 75.5% (Block 12).

Conversely, DMoL ($\alpha = 0.5$) acts as a mathematical “brake”. By intentionally delaying early convergence, it forces a residual fitting paradigm that explicitly preserves high-dimensional feature diversity. This delayed gratification enables the network to fully utilize its depth, ultimately achieving 81.5% (+6.0% over the greedy baseline). Thus, the KD mechanism genuinely **enhances** the final representation by preventing greedy collapse.

Table 9. Block-wise Test Accuracy (%) on CIFAR-100 (ViT-Small). DMoL postpones early gains to achieve a significantly higher final capacity.

Strategy	Blk 2	Blk 4	Blk 6	Blk 8	Blk 10	Blk 12 (Final)
$\alpha = 1.0$ (Greedy)	53.2%	68.4%	73.1%	74.8%	75.3%	75.5%
$\alpha = 0.8$ (Weak KD)	49.5%	65.6%	74.2%	77.5%	79.1%	80.2%
$\alpha = 0.5$ (DMoL)	35.8%	56.2%	75.4%	78.9%	80.6%	81.5%
$\alpha = 0.2$ (Excessive KD)	18.4%	31.5%	46.2%	59.8%	67.4%	72.1%

G.3. GCN Performance Breakdown: Auxiliary Classifiers vs. DMoL’s Mechanism

To confirm that DMoL’s substantial performance gains in deep GNNs (+37.5% on Cora) do not stem solely from a “free ride” provided by adding auxiliary classifiers, we benchmarked DMoL against GoogLeNet-style intermediate supervision and

Table 10. Block-wise Test Accuracy (%) on Tiny-ImageNet (ViT-Small).

Strategy	Blk 2	Blk 4	Blk 6	Blk 8	Blk 10	Blk 12 (Final)
$\alpha = 1.0$ (Greedy)	42.1%	55.3%	59.6%	61.2%	61.8%	62.0%
$\alpha = 0.5$ (DMoL)	28.6%	46.1%	61.4%	64.8%	66.7%	68.0%

Deep Supervision (DeepSup). As shown in Table 11, while intermediate supervision provides some benefits over standard BP, it is insufficient to explain DMoL’s gains. DMoL’s decoupled cooperative refinement explicitly forces progressive belief disentanglement, significantly outperforming mere auxiliary supervision.

Table 11. GCN Performance Breakdown. Test accuracy (%) comparing standard BP, GoogLeNet-style supervision, Deep Supervision, and DMoL on 8-layer GCNs.

Dataset	BP	GoogLeNet-style	DeepSup	DMoL (Ours)
Cora	37.06 ± 9.97	41.52 ± 5.80	70.12 ± 11.77	80.26 ± 0.98
PubMed	64.30 ± 4.79	66.24 ± 6.61	71.94 ± 9.87	77.14 ± 0.39
Actor	25.72 ± 0.53	31.24 ± 3.35	25.96 ± 0.63	27.76 ± 0.56